

Compiler Autotuning through Multiple-phase Learning

MINGXUAN ZHU and DAN HAO, Key Laboratory of High Confidence Software Technologies (Peking University), Ministry of Education; School of Computer Science, Peking University, Beijing, China
JUNJIE CHEN, Tianjin University, Tianjin, China

Widely used compilers like GCC and LLVM usually have hundreds of optimizations controlled by optimization flags, which are enabled or disabled during compilation to improve the runtime performance (e.g., small execution time) of the compiler program. Due to the large number of optimization flags and their combination, it is difficult for compiler users to manually tune compiler optimization flags. In the literature, a number of autotuning techniques have been proposed, which tune optimization flags for a compiled program by comparing its actual runtime performance with different optimization flag combinations. Due to the huge search space and heavy actual runtime cost, these techniques suffer from the widely recognized efficiency problem. To reduce the heavy runtime cost, in this article we propose a lightweight learning approach that uses a small number of actual runtime performance data to predict the runtime performance of a compiled program with various optimization flag combinations. Furthermore, to reduce the search space, we design a novel particle swarm algorithm that tunes compiler optimization flags with the prediction model. To evaluate the performance of the proposed approach, CompTuner, we conduct an extensive experimental study on two popular C compilers, GCC and LLVM, with two widely used benchmarks, cBench and PolyBench. The experimental results show that CompTuner significantly outperforms the six compared techniques, including the state-of-the-art technique BOCA.

CCS Concepts: • **Software and its engineering** → **Compilers**;

Additional Key Words and Phrases: Compiler, compiler autotuning, multiple phase learning, particle swarm optimization

ACM Reference Format:

Mingxuan Zhu, Dan Hao, and Junjie Chen. 2024. Compiler Autotuning through Multiple-phase Learning. *ACM Trans. Softw. Eng. Methodol.* 33, 4, Article 100 (April 2024), 38 pages. <https://doi.org/10.1145/3640330>

1 INTRODUCTION

Compilers play an important role in software development, especially program execution. A widely used compiler like GCC or LLVM usually has hundreds of optimizations, e.g., *function inlining* and *useless code removal*, which are controlled by optimization flags. During compilation,

This work was supported by the National Natural Science Foundation of China under Grant No. 62232001, National Natural Science Foundation of China under Grant No. 62322208, and CCF Young Elite Scientists Sponsorship Program (by CAST). Authors' addresses: M. Zhu and D. Hao (Corresponding author), Key Laboratory of High Confidence Software Technologies (Peking University), Ministry of Education; School of Computer Science, Peking University, Beijing, China, No. 5 Yiheyuan Road, 100871; e-mails: zhumingxuan@stu.pku.edu.cn, haodan@pku.edu.cn; J. Chen, Tianjin University, College of Intelligence and Computing, Tianjin, China, No. 135 Yaguan Road, 300350; e-mail: junjiechen@tju.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1049-331X/2024/04-ART100

<https://doi.org/10.1145/3640330>

these optimizations can be enabled or disabled, resulting in various compiled programs with different runtime performance [37, 45, 51]. For example, the compiler optimization flag *-floop-unroll-and-jam* can perform external loop unfolding and internal loop fusion to optimize loop statements in a program. That is, compiler optimization flags have a significant impact on the runtime performance of a compiled program.

On the other hand, the same optimization does not always result in the same performance improvement when being applied to different programs. Although widely used compilers like GCC and LLVM have recommended optimization settings such as -O1, -O2, and -O3, such optimization settings do not guarantee to always improve runtime performance for any program.

Therefore, it is necessary to choose *specific* compiler optimization settings for various programs, especially for the programs that need a long time to run and that optimization flags have a significant impact on. In particular, a compiler optimization setting (i.e., compiler tuning) consists of selecting the right set of compiler optimizations (also known as “flag selection” [12, 21] or “phase selection” [28]) and deciding the order in which these optimizations are applied (also known as “phase ordering” [5, 27]). In this article, we focus on the former one, the flag selection problem.¹ Considering the large number of optimizations and their exponential combinations, it is difficult for compiler users to understand the impact of each compiler flag on the program and manually choose compiler optimization settings for a program.

To alleviate human efforts in setting compiler optimizations, in the literature a number of compiler autotuning techniques focusing on *flag selection* have been proposed, which are divided into two categories [4, 6], supervised learning-based and unsupervised learning-based techniques. In particular, unsupervised learning-based techniques [2, 10, 21, 26, 32, 35, 39, 42, 43] were first proposed to address compiler autotuning. Typically, for a program under compiling, the unsupervised learning-based techniques explore its searching space of optimization flags through some searching strategies (e.g., hill-climbing-based algorithm [2] and genetic algorithm [21, 26, 35, 43]) and select based on the actual runtime performance of the program compiled with the corresponding flag combinations. During the searching process, these techniques usually generate a large number of flag combinations, whose corresponding runtime performance of the target program has to be collected via compilation and execution. That is, these techniques are very time-consuming.

Recently, BOCA [12] has been proposed to reduce the search space of optimization combinations through impactful flags, which are identified through a learning-based prediction model. In detail, BOCA selects only the instances with better performance to build the prediction model. With these biased instances, the prediction model BOCA produced may have low accuracy, resulting in impactful flag identification errors.² These accumulated errors may lead to poor optimization results. For example, for the program “*symm*” [41] in polyBench, BOCA identifies eight impactful flags in total. One of these eight flags is related to floating-point arithmetic structure and two of these eight flags are related to branch arithmetic structure, but this program has no floating-point arithmetic or branch structures at all.

To alleviate the time cost and performance³ issues of existing compiler autotuning techniques, in this article, we propose a multiple-phase learning-based compiler autotuning technique, *CompTuner*, which first builds a prediction model through multiple phases of learning and searches for a desired flag combination through another round of learning. In particular, CompTuner first builds

¹In our article, we use a flag combination to represent the result of a flag selection technique, encompassing all enabled flags.

²As BOCA performs bias selection of the impactful flags, inaccurate identification of impactful flags may lead to selecting the flag combinations with poor performance.

³In this article, the performance of a compiler autotuning technique refers to the runtime speedup of a program compiled with the flag combinations selected by the corresponding technique over the compiler default -O3 optimization.

a prediction model through multiple phases of learning, each of which uses only a very small number of carefully selected data (i.e., flag combinations and the corresponding actual runtime performance). This lightweight learning process alleviates the cost issue of existing supervised learning-based techniques without scarifying the accuracy of a prediction model. Then, CompTuner searches for a flag combination with good runtime performance through an improved particle swarm optimization algorithm [29, 40, 48], which balances the local exploitation and global exploration considering performance diversity. The proposed CompTuner is different from the latest work, BOCA, in both building prediction models and the searching process. In particular, BOCA builds a prediction model through selected flag combinations with the best performance, whereas CompTuner builds a prediction model by considering flag combinations with performance diversity. In the search process, BOCA narrows the search space by identifying impactful flags, while CompTuner searches with an improved particle swarm algorithm. BOCA uses a prediction model to identify impactful flags and then narrows the search space by these impactful flags. That is, the prediction model with low accuracy may result in errors in impactful flag identification. With these accumulated errors, BOCA is unable to accurately reduce the optimization space, which in turn leads to the failure to select a better compiler optimization flag combination.

To evaluate the performance of our approach, we conducted an experiment on two widely used compilers, GCC 8.3.0 and LLVM 9.0.0, using 20 programs from two widely used benchmarks, cBench [11] and PolyBench [41]. Our evaluation contains two parts: a study on the overall effectiveness of our proposed CompTuner and its ablation study. In these studies, we run our approach and six compared techniques within a time limit to obtain their optimization results for a program and to compare them. Since the memory of our system is large enough that the program does not crash due to the lack of memory problem, we do not set a memory limit.

From the overall study, the flag combinations selected by CompTuner are always better than the default -O3 optimization of GCC and LLVM. Moreover, given a time limitation for compiler autotuning, the flag combinations selected by CompTuner significantly outperform all the six representative compiler autotuning techniques [3, 7, 9, 12, 13, 21]. Moreover, our proposed CompTuner achieves the best optimization results on many cases (i.e., seven programs on GCC and six programs on LLVM). Besides, we manually analyze optimization flags selected by BOCA and CompTuner, as well as the best flag combination obtained through a searching process. From the analysis, BOCA recommends different optimization flags for a program in different trials, indicating that BOCA is not stable. Moreover, the optimization flags selected by CompTuner and the best flag combinations have a large overlap, indicating that CompTuner is promising in selecting proper optimization flags.

Furthermore, for the ablation study, we have compared the model construction process in the first stage and the flag combination selection process in the second stage, respectively. For the model construction process, we have compared CompTuner's construction way (i.e., selecting the instances with performance difference to construct a prediction model), the construction way of selecting only the flag combinations with the highest expected improvement, and BOCA's construction way. For the flag combination selection process, we have compared CompTuner's selection way (i.e., selecting the flag combinations by the improved particle swarm optimization algorithm) and the selection way of using impactful flags to reduce the searching space. From the ablation study, the prediction model built by CompTuner has high accuracy. Since CompTuner predicts the performance of compiler flag combinations via a prediction model, the model with a lower prediction error rate will make the prediction results closer to the real performance, which ensures that the optimization process is not biased. Moreover, the improved particle swarm optimization algorithm does improve the performance of CompTuner by selecting flag combinations with diversity.

Among the six experimental cases we chose, CompTuner achieves better speedups on four cases, while the variant technique⁴ only performs better on two cases.

The contributions of this article are summarized as below:

- A novel multiple-phase learning-based compiler autotuning approach, CompTuner, which builds a prediction model with small numbers of selected instances and selects compiler flag combinations through improved particle swarm optimization
- An extensive experiment on GCC and LLVM using two compiler optimization benchmarks, cBench and PolyBench, which demonstrates that CompTuner is effective and promising
- A reproducible package available at <https://github.com/TOSEM202210/compTuner>

The remainder of this article is organized as follows. Section 2 introduces the approach details. Section 3 presents the experimental setup. Section 4 presents the experimental results and analysis. Section 5 presents the threats to validity. Section 6 presents the discussion from three aspects. Section 7 briefly reviews the related work. Finally, Section 8 concludes the whole article.

2 TECHNIQUE

A compiler is to translate a program in one programming language into another programming language, often resulting in an executable program. To improve the compiled-code size or runtime performance, a traditional compiler like GCC or LLVM usually has hundreds of compiler optimization flags. For example, the GCC's optimization flags *-floop-interchange*, *-floop-unroll-and-jam*, and *-fmove-loop-invariants* are to optimize the loop part of a program, and *-fcse-follow-jumps* and *-fguess-branch-probability* are to optimize the branch part of a program. In particular, an optimization flag specifies a kind of program transformation for some program structure, whose value is usually 0/1⁵ (i.e., 0 represents that the corresponding optimization flag is disabled and 1 represents that the corresponding optimization flag is enabled).

For ease of presentation, we formalize the problem this article targets as below. Given a compiler, its set of optimizations can be represented by $O = \{o_1, o_2, \dots, o_n\}$, where n represents the number of optimizations and the value of o_i (where $1 \leq i \leq n$) represents whether the corresponding optimization flag is enabled or disabled (i.e., o_i is set 1 or 0). A specific setting on these optimization flags can be represented by a set of enabled optimization flags, which is called a flag combination. The size of the whole optimization space (i.e., including all flag combinations) is 2^n , which is very huge. For example, GCC 8.3.0 has 106 optimizations, and the size of the optimization space is 2^{106} . On the other hand, enabling an optimization flag does not always improve the runtime behavior of a compiled program, and the same optimization flag may have different effects on various compiled programs. Due to the huge optimization space and diverse performance of each optimization flag, manual compiler tuning is impossible, and thus in this article we target compiler autotuning, i.e., automatically identifying a specific flag combination for a compiled program with the goal of runtime performance improvement. In particular, this article focuses on short execution time, which is the mostly used compiler optimization goal [12], and we leave other compiler optimization goals like small executable code as our future work.

2.1 Approach Overview

The existing compiler autotuning techniques usually find a desired flag combination by comparing the *actual* execution time of the compiled program with different flag combinations (i.e., setting various values to the optimization flags in O), which suffers from a performance problem due to

⁴That is, a variant technique uses impactful flags to reduce the search space instead of particle swarm optimization.

⁵A few optimization flags have other values instead of 0 and 1. For example, *-falign-functions* can be set to 24 or 32, which represents the align position of a function. Following all previous work [12, 13], we focus on optimization flags whose values are only 0 or 1 in this article.

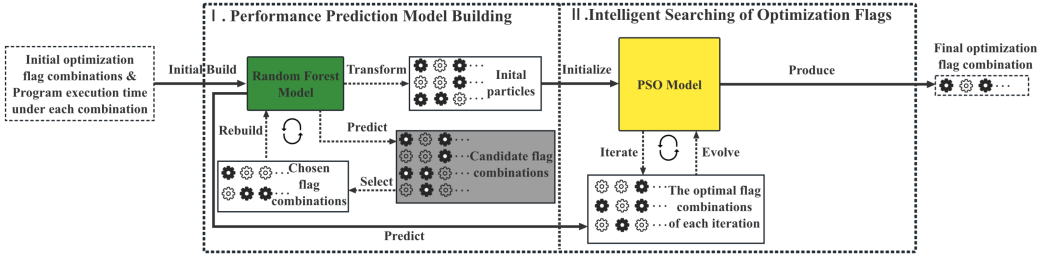


Fig. 1. Workflow of CompTuner.

the huge search space and execution time. To address the performance problem, in this article, we propose a multiple-phase learning-based approach, CompTuner, which alleviates the two performance issues (i.e., long execution time and huge search space) through two learning phases.

Figure 1 shows the overview of CompTuner, which consists of prediction model building and intelligent searching.

- *Prediction model building* is to learn a prediction model on the runtime performance of a compiled program with various flag combinations by using only a small number of training instances. In particular, we propose a lightweight learning approach, which builds a prediction model through initial learning on randomly generated flag combinations and enhances the prediction accuracy through several phases of learning based on flag combinations with diversity. More details are referred to Section 2.2.
- *Intelligent searching* is to search for a desired flag combination of a target program through an improved particle swarm optimization algorithm [29, 40, 48]. In particular, we transform the compiler autotuning problem into a particle swarm optimization algorithm where a flag combination is regarded as a particle and then expand the search space by refining the parameters in the particle swarm optimization algorithm based on the performance similarity of generated flag combinations. More details are referred to Section 2.3.

2.2 Performance-prediction Model Building

To improve compiler autotuning efficiency, the proposed CompTuner predicts the runtime performance of a compiled program with a flag combination instead of actual execution.

Intuitively, a prediction model with high accuracy is expected to be learned with many instances (i.e., the actual runtime performance of a compiled program with various flag combinations), but labeling so many instances (i.e., executing the compiled program with various flag combinations) is undoubtedly time consuming. To address this issue, we propose a lightweight learning approach that builds a prediction model through several phases of learning: initial learning builds a prediction model with a small number of randomly generated instances, and enhancement learning improves the accuracy of the prediction model through several phases of learning by using carefully selected instances (with performance diversity). More specifically, the instances used in this learning process are flag combinations whose labels are actual runtime performance of the compiled program with the corresponding flag combinations. In particular, the features of the training data are compiler optimization flags, each of which is set to 0 or 1, representing whether the corresponding optimization flag is off or on. Note that we do not build a unified prediction model for all programs, but an individual prediction model for a program.⁶

⁶In the evaluation, for each program in the benchmark, we build a prediction model and search for its compiler flag combinations.

2.2.1 Initial Learning. In initial learning, CompTuner builds a prediction model with a very small number of random generated data. In particular, CompTuner first randomly generates a small number of flag combinations and collects the runtime performance of the compiled program with each flag combination (i.e., input data of the “Initial Build” operation in Figure 1), which are regarded as the training data of initial learning. Note that in our experiment the number of training data used in initial learning is set to 2. Then CompTuner builds a prediction model based on these data with a supervised learning algorithm (i.e., random forest in this article), as the random forest algorithm [12] combines the decisions from multiple decision trees and performs well on data with high-dimensional features, while a compiler flag combination has around 100 optimization flags.

2.2.2 Enhancement Learning. As initial learning builds a prediction model with a very small number of data, its prediction may be far from accurate. To address this problem, CompTuner further selects a few more flag combinations with more runtime performance diversity, which are used to further enhance the accuracy of the prediction model through several rounds of the learning process. Since CompTuner builds a prediction model with a small number of training data, it is a lightweight learning approach.

In enhancement learning, CompTuner randomly generates a set of candidate flag combinations and selects flag combinations with diverse runtime performance, which are fed to train a prediction model. For a target program with a flag combination, the multiple trees of the random forest model generated by CompTuner give various predicted values on its runtime performance, and thus CompTuner combines the predicted performance resulting from different trees following Bayesian optimization [12, 20, 47, 50]. That is, CompTuner combines the mean and variance on these predicted values to get the overall performance (denoted as EI) of the flag combination as Equation (1). The “Select” operation in Figure 1 is conducted based on the EI value of each candidate combination.

$$EI(combination) = (f' - \mu(combination))\Phi(Z) + \sigma(combination)\phi(Z) \quad (1)$$

$$\text{where } Z = \frac{f' - \mu(combination)}{\sigma(combination)},$$

where f' is the minimum execution time of flag combinations to build the prediction model. For any $combination$, $\mu(combination)$ and $\sigma(combination)$ represent the mean and variance of the prediction results of the trees in the random forest model. $\Phi(Z)$ is the cumulative distribution and $\phi(Z)$ is the probability density of standard Gaussian distribution on combinations' performance.

To improve the accuracy of the prediction model, in each cycle of enhancing learning, CompTuner selects flag combinations with performance diversity, i.e., considering good/bad runtime performance and randomness. Following this intuition, among the set of randomly generated combinations, besides selecting the flag combination with the largest EI value, CompTuner checks the prediction accuracy of the combination. If the accuracy does not reach the standard (set to 0.95 in the experiment), CompTuner selects another flag combination with small EI values with high probability. In particular, CompTuner first sorts all generated combinations based on the descent order of their EI values and assigns a probability to each flag combination by guaranteeing that the flag combination with small EI values has large probability. Finally, CompTuner randomly selects a flag combination among all generated combinations, which might be a flag combination with low performance [30, 38, 49].

With the selected flag combinations (including their actual runtime performance), CompTuner trains another prediction model, shown by the “Rebuild” operation in Figure 1. CompTuner repeats this enhancement learning process until the prediction model is accurate enough. That is, CompTuner achieves the desired accuracy of the prediction model through initial learning and

several phases of enhancement learning. In the evaluation, the termination criterion of the learning process (i.e., the desired prediction accuracy) is set to 0.96.⁷ With such a build-tune process, CompTuner constructs a more accurate prediction model and selects better starting data for the intelligent searching process.

2.2.3 Algorithm Details. More details of this prediction model-building algorithm are given in Algorithm 1.

The algorithm to build a performance-prediction model has the following input parameters: *IniSize* representing the number of instances for the initial model, *Acc* representing the prediction accuracy threshold, and *N* representing the number of instances for the final model. Lines 2–3 initialize two vectors, *Coms* and *PerofComs*, which represent the instances used for prediction model building. Lines 5–12 generate instances for the initial performance-prediction model by randomly generating 0-1 combinations. In particular, Line 6 obtains each combination’s performance by actual execution. Line 13 builds a prediction model with a random forest model.

Lines 15–39 tune the prediction model by selecting new instances. In particular, lines 17–24 select a new combination to tune the prediction model. Lines 18–19 obtain the performance of each candidate combination and select the combination with the best performance. Line 18 uses the prediction model to predict the candidate combinations. Lines 25–35 select another combination according to performance diversity. Line 25 obtains the accuracy of the selected combination by comparing its actual performance and predicted performance. Lines 27–35 check the accuracy and decide whether to select a new sequence. Lines 28–31 select a new combination by giving lower-performing combinations a larger probability of being selected. Lines 37–38 decide whether this algorithm stops by checking the average prediction accuracy for the selected combinations. Lines 36 retrains the prediction model by the new instances.

2.3 Intelligent Searching of Optimization Flags

To select a desired compiler flag combination for the target program, CompTuner reuses the data (i.e., which are obtained by the “Transform” operation in Figure 1) in building the prediction model as the starting point of the following searching process and designs a particle swarm optimization algorithm. To avoid the local optimum problem, we further expand the search space by refining the parameters of the particle swarm optimization algorithm, which influence the exploring speed, resulting in an improved particle swarm optimization algorithm. In the following paragraphs, we first introduce how we solve the compiler autotuning problem through a particle swarm optimization algorithm and then how to improve this algorithm.

2.3.1 Particle Swarm Optimization. The particle swarm optimization algorithm [29, 40, 48] is a representative search algorithm, which accomplishes an optimization process by simulating the foraging behavior of birds and is widely used due to its extraordinary performance in balancing the local and global optimum. In our improved particle swarm optimization algorithm, we regard each flag combination *O* as a *particle* whose initial movement velocity vector is represented by V_i and initial position vector (representing each flag combination’s enable/disable choices) is represented by X_i , and its following movement velocity vector is updated based on Equation (2):

$$V_i^{(t+1)} = \omega \times V_i^{(t)} + c_1 \times rand_1^{(t)} \times (pBest_i^{(t)} - X_i^{(t)}) + c_2 \times rand_2^{(t)} \times (gBest^{(t)} - X_i^{(t)}), \quad (2)$$

⁷For some target programs, this accuracy may be hard to achieve. In such cases, to avoid training a prediction model with unbearable time cost (reflected by too much data), we also stop the training process when the number of training data exceeds 50.

ALGORITHM 1: Performance-prediction Model Building

```

1: procedure BUILDING(IniSize, Acc, N)
2:   Coms  $\leftarrow$  []
3:   PerofComs  $\leftarrow$  []
4:   // Lines 5–13 build the initial performance prediction model
5:   while Sizeof(Coms) < IniSize do
6:     com  $\leftarrow$  random(0,1)number_of_flags
7:     if com not in Coms then
8:       per  $\leftarrow$  Performance(com)
9:       Coms.append(com)
10:      PerofComs.append(com)
11:     end if
12:   end while
13:   model  $\leftarrow$  RandomForestModel(Coms, PerofComs)
14:   // Lines 15–39 tune the performance prediction model
15:   while Sizeof(Coms) < N do
16:     // Lines 17–24 select combinations with highest EI from candidates
17:     candidates  $\leftarrow$  Set(random(0,1)number_of_flags)
18:     model.predict(candidates)
19:     com  $\leftarrow$  selectFromcandidatesByEI
20:     if com not in Coms then
21:       per  $\leftarrow$  Performance(com)
22:       Coms.append(com)
23:       PerofComs.append(per)
24:     end if
25:     acc  $\leftarrow$  Accuracy(com)
26:     // Lines 27–35 select combinations by performance distribution from candidates
27:     if acc < Acc then
28:       com_new  $\leftarrow$  selectFromcandidatesByValueDistribution
29:       while com_new in Coms do
30:         com_new  $\leftarrow$  selectFromcandidatesByValueDistribution
31:       end while
32:       per  $\leftarrow$  Performance(com_new)
33:       Coms.append(com_new)
34:       PerofComs.append(per)
35:     end if
36:     model  $\leftarrow$  RandomForestModel(Coms, PerofComs)
37:     if AverageAcc(Coms) > Acc then break
38:   end if
39: end while
40: end procedure

```

where ω is the inertia weight, regulating the search range of the solution space. c_1, c_2 are acceleration constants, which regulate the step size of the exploration. An increase in c_1 indicates an enhanced local search, and an increase in c_2 indicates an enhanced global search. $pBest$ is the historical optimal position vector of each particle, and $gBest$ is the global historical position vector of all particles. $rand_1$ and $rand_2$ are randomly generated values between 0 and 1 in each iteration, which perturb the particle's velocity update. In particular, $X_i^{(t)}$ is the particle's position in the t_{th} iteration and $V_i^{(t)}$ is the particle's movement velocity in the t_{th} iteration. Through these variables, CompTuner updates each particle's movement velocity vector for the $t + 1_{th}$ iteration, as well as

the particles' position through the updated velocity vector, as shown in Equations (3) and (4):

$$s_{i,j}^{(t)} = \frac{1}{1 + e^{-v_{(i,j)}^{(t)}}}, \quad (3)$$

$$x_{i,j}^{(t)} = \begin{cases} 1, & rand() < s_{i,j}^{(t)} \\ 0, & rand() \geq s_{i,j}^{(t)}. \end{cases} \quad (4)$$

Due to the position vector of the particle consisting of 0 and 1, CompTuner adjusts the position vector of each particle by introducing a temporary vector S_i . In the t_{th} iteration, the values of the temporary vector $S_i^{(t)}$ of each particle are obtained by the updated movement velocity vector $S_i^{(t)}$ via the sigmoid function as Equation (3), which turns the value range of a temporary vector to be $[0, 1]$.

Through the temporary vectors, CompTuner updates the position vector for each particle according to Equation (4). For particle i , if the j_{th} element $s_{i,j}$ of the temporary vector S_i is greater than a random value (whose value range is $[0, 1]$), the j_{th} element $x_{i,j}$ of the position vector X_i is updated to 1; otherwise, it is updated to 0. In this way, CompTuner completes the update for the velocity vector and the position vector of each particle in this iteration, shown by the "Evolve" operation in Figure 1.

In each iteration, CompTuner uses the prediction model (generated in Section 2.2) to predict the runtime performance (i.e., execution time in this article) of a generated flag combination (i.e., a particle's position vector) rather than actual execution, to reduce the time spent during compiler autotuning, shown by the "Predict" operation in Figure 1. In particular, in each iteration CompTuner updates the optimal position $gBest_i$ of each particle and the optimal position $pBest$ of all particles by the predicted runtime performance of a flag combination (i.e., the position vector of each particle).

2.3.2 Improvement on Particle Swarm Optimization. To avoid falling into local optimization, CompTuner further expands the search space of flag combinations by using combinations with good performance and combinations with bad performance in different ways. In particular, CompTuner identifies the optimal sequence among all the combinations generated during each iteration based on their predicted performance and classifies all these combinations into two sets (i.e., a set of combinations with good performance and a set of combinations with bad performance) based on whether a combination achieves similar performance as the optimal combination. Moreover, CompTuner decides whether a flag combination achieves similar performance as the optimal combination based on their combination similarity since the acceleration effect of similar flag combinations also has certain proximity. In particular, CompTuner uses Equation (5) (i.e., the cosine similarity of two vectors X_i and X_j [55]) to calculate the similarity between a combination (denoted as X_i) and the optimal combination (denoted as X_j), and represents their s_{th} compiler optimization flag (0 or 1) by $x_{i,s}$ and $x_{j,s}$:

$$sim(X_i, X_j) = \frac{\sum_{s=1}^n (x_{i,s} \times x_{j,s})}{\sqrt{\sum_{s=1}^n (x_{i,s})^2} \times \sqrt{\sum_{s=1}^n (x_{j,s})^2}}. \quad (5)$$

Based on this formula, CompTuner classifies the generated combinations into a set of combinations with strong similarity with the optimal combination (i.e., set of sequences with good performance, denoted as $part_1$) and a set of combinations with poor similarity with the optimal combination (i.e., set of combinations with bad performance, denoted as $part_2$). Furthermore, if the optimal combination's runtime performance generated during this iteration is better than existing optimal combinations' runtime performance, CompTuner increases the c_1 value for particles in $part_1$ to strengthen the local search, trying to find a better result among the current optimal value,

ALGORITHM 2: Intelligent Searching

```

1: procedure SEARCHING(IniPars, cominitial, sinitial)
2:   bestPerofEach  $\leftarrow$  PerofEachParofIniPars
3:   bestComofEach  $\leftarrow$  ComofEachParofIniPars
4:   bestPerofAll  $\leftarrow$  sinitial
5:   bestComofAll  $\leftarrow$  cominitial
6:   velofEach  $\leftarrow$  RandomlyGenerated
7:   // Lines 8–9 update the particles to generate new combinations
8:   for i  $\leftarrow$  1 to M do
9:     if i = 1 then
10:       NewPars  $\leftarrow$  Evolution(IniPars, velofEach)
11:       // Lines 13–22 predict the particles' position vector and update each particle
12:     else
13:       prePer  $\leftarrow$  []
14:       for j  $\leftarrow$  1 to sizeof(IniPars) do
15:         prePer[j]  $\leftarrow$  model.predict(NewPars[j])
16:         if prePer[j]  $\leq$  bestPerofEach[j] then
17:           bestPerofEach[j]  $\leftarrow$  prePer[j]
18:           bestComofEach[j]  $\leftarrow$  NewPars[j]
19:         else
20:           continue
21:         end if
22:       end for
23:       // Lines 23–33 update the particles to generate new combinations
24:       CurrentBestPer  $\leftarrow$  best(bestPerofAll)
25:       if CurrentBestPer > bestPerofAll then
26:         bestPerofAll  $\leftarrow$  CurrentBestPer
27:         bestComofAll  $\leftarrow$  best(bestComofEach)
28:         NewPars  $\leftarrow$  Evolution(NewPars)
29:       else
30:         NewPars1, NewPars2  $\leftarrow$  divide(NewPars)
31:         NewPars1  $\leftarrow$  Evolution(NewPars1)
32:         NewPars2  $\leftarrow$  Evolution(NewPars2)
33:         NewPars  $\leftarrow$  Combine(NewPars1, NewPars2)
34:       end if
35:     end if
36:   end for
37:   (comfinal, sfinal)  $\leftarrow$  (bestComofAll, bestPerofAll)
38:   return (comfinal, sfinal)
39: end procedure

```

and increases the c_2 value for particles in $part_2$ to strengthen the global search, trying to get the desired result in the bigger search space.

2.3.3 Algorithm Details. More details of this improved particle swarm optimization algorithm are given by Algorithm 2.

The algorithm of performance-prediction model building has input parameters *IniPars* representing the dataset used to build a prediction model, *seq_{initial}* representing the combination with the best performance in building a prediction model, and *s_{initial}* representing the best performance in building a prediction model. Lines 2–6 initialize particles for the searching process. Lines 2–5 initialize the optimal position vector (i.e., combination) and performance (i.e., the execution time

of the target program) of each particle by *InitialPars*, and the optimal position vector and performance of all the particles by *seq_{initial}* and *s_{initial}*. Line 6 randomly generates a velocity vector for each particle.

Lines 8–37 perform an iterative update of the entire particle swarm, where the position vector of each particle continuously changes during searching (i.e., generating new combinations) and the new combinations are predicted by the generated prediction model. CompTuner continuously selects the combination with the best performance until reaches the termination condition. Lines 9–10 update each particle’s position vector in the first round of evolution by using a randomly generated velocity vector according to Equations (3) and (4) in Section 2.3.1. In the subsequent iterations, Lines 12–22 update the best position and performance of each particle. Line 15 predicts the new position vector of each particle updated in the previous iteration, and then Lines 16–21 update the optimal position vector (i.e., compiler flag combination) and optimal performance of each particle.

Lines 24–35 update the position vector of each particle again in the new evolution. To avoid the search process falling into a local optimum, CompTuner provides two ways to update. Line 24 first obtains the optimal performance of all particles in this iteration. If this performance is better than the previous optimal performance of all particles, Lines 25–28 update the optimal performance and the optimal position of all particles, as well as the particles’ position vectors as Equations (2)–(4). Conversely, Lines 29–33 divide all the particles into two groups based on the optimal combination of this iteration as Equation (5), update the two groups in different ways (i.e., adjusting the values of c_1 and c_2 in Equation (2)), and finally merge the particles in the two groups. Lines 34–35 obtain the desired compiler combination and the execution time of the compiled program under the flag combination.

3 EXPERIMENTAL SETUP

To evaluate the performance of CompTuner, this experiment is designed to answer two research questions:

- *RQ1: Effectiveness.* How does CompTuner perform compared to the state-of-the-art compiler autotuning techniques?
- *RQ2: Ablation analysis.* How does each component of CompTuner contribute to the performance?

3.1 Dataset

In this experiment, we choose two widely used C compilers, GCC 8.3.0 [22] and LLVM 9.0.0 [31], as target compilers, which have been used in numerous studies [12, 16–18].⁸ Like the state-of-the-art work does [12], we use the same programs from two widely used benchmarks, cBench [11] and PolyBench [41], as target programs.

Table 1 presents the statistics of these compiled programs. The first 10 programs are from PolyBench and the others are from cBench. That is, in this experiment, we have 20 compiled programs that cover a wide range of practical functions, ranging from 200 to 26,000 lines of code.⁹

⁸To verify the effectiveness of CompTuner on the latest compilers, we also experiment with GCC 12.2.0 and LLVM 15.0.0 and discuss their results in Section 6.2.

⁹To verify CompTuner’s performance on a wider range of programs, we conduct another experiment on the remaining 20 programs of PolyBench and the remaining 18 programs of cBench. As the entire time cost of this added experiment is very costly, in the added experiment, we apply each compiler autotuning technique to each of these programs only once and present the corresponding results in the appendix. Besides, the results on these remaining programs of cBench and PolyBench are consistent with the conclusion drawn from this section.

Table 1. Statistics of Programs

ID	Program	Function	Lines
P1	correlation	Compute correlation	248
P2	covariance	Compute covariance	218
P3	symm	Symmetric matrix multiplication	231
P4	2mm	2 matrix multiplications	252
P5	3mm	3 matrix multiplications	267
P6	cholesky	Cholesky decomposition	212
P7	lu	LU decomposition	210
P8	nussinov	Predict RNA folding	569
P9	heat-3d	Heat Equation over 3D space	211
P10	jacobi-2d	Jacobi-style stencil computation	200
C1	automotive_bitcount	Testing bit manipulation	954
C2	automotive_susan_e	Edge image recognition	2,129
C3	automotive_susan_c	Corner image recognition	2,129
C4	automotive_susan_s	Image smoothing	2,129
C5	consumer_tiff2rgba	TIFF image conversion	22,321
C6	consumer_jpeg_c	Image compression	26,950
C7	office_rsynth	Text-to-speech synthesis	5,412
C8	security_sha	Secure hash algorithm	297
C9	bzip2e	File compression	7,200
C10	telecom_adpcm_c	Pulse Code Modulation	389

3.2 Compared Techniques

The existing compiler autotuning techniques are classified into unsupervised learning-based techniques and supervised learning-based techniques [4, 6].

Unsupervised leaning-based techniques select a compiler flag combination with good performance via a specific search strategy and actual execution of the target program in the optimization space. We consider two representative unsupervised leaning-based techniques, *random iterative optimization (RIO)* [13] (which is a simple unsupervised learning-based technique) and *genetic algorithm-based technique (GA)* [21] (which is the state-of-the-art unsupervised learning-based technique). Besides, in this study, we also consider *OpenTuner*,¹⁰ which is a general tuning framework in Github, not specialized for compiler autotuning.

Supervised learning-based techniques learn performance knowledge from some existing flag combinations, then generate new compiler flag combinations using some methods and predict them, and finally select an ideal compiler flag combination. We consider three representative supervised learning-based techniques, *TPE* [9], *BOCA* [12], and *COBYAN* [7]. In particular, TPE is actually a general but state-of-the-art Bayesian optimization-based technique, not specific for compiler autotuning. BOCA is a state-of-the-art compiler autotuning technique. COBYAN is a compiler autotuning framework using Bayesian Networks.

To alleviate the influence of randomness resulting from the proposed technique and compared techniques, we repeat each technique five times.

3.3 Implementation

We implement CompTuner in Python based on numpy [36] and scikit-learn [46]. We adopt the implementation of the Random Forest Model in scikit-learn with default parameter settings and

¹⁰<https://github.com/jansel/opentuner>

Table 2. Cost of CompTuner in Building a Prediction Model (Number Stands for Number of Training Instances and Time Stands for Time (in Seconds) to Build a Prediction Model)

ID	(Number/time(s))	ID	(Number/time(s))	ID	(Number/time(s))
P1	44/1,836	P2	46/2,233	P3	41/1,812
C1	41/2,257	C2	35/1,944	C3	38/2,039

set parameter values for the proposed particle swarm optimization algorithm based on some trials. That is, the parameters c_1 , c_2 , and w are set to be 2, 2, and 0.6.

In implementing the compared techniques, we directly use their code if they provide a reproducible package (i.e., for techniques TPE, BOCA, and OpenTuner); otherwise, we re-implement the techniques (i.e., GA, RIO, and COBYAN) strictly following the description in the papers.

The experiment is performed on a workstation with Intel(R) Xeon(R) Gold 5218R CPU @ 2.10 GHz, 377.8 G memory, and Ubuntu 16.04.6 LTS operating system.

3.4 Measurement

As previous work did [12], in this experiment, we measure a compiler autotuning technique's runtime performance improvement by comparing against the compiler's default optimization. In particular, GCC and LLVM have -O1/O2/O3/Ofast optimization settings and we use their -O3 optimization settings as the compiler's default optimization in the study since they are the highest default optimization settings of these compilers and widely used by existing compiler tuning work [12].

In order to obtain the runtime performance improvement of a compiler autotuning technique, for any C program p , we compile p with the flag combination selected by a compiler autotuning technique and with the default compiler optimization -O3 separately, resulting in two compiled programs, p_r and p_d . Then we run the two programs, p_r and p_d , and calculate the *runtime performance acceleration speedup* of p (achieved by a compiler autotuning technique) through dividing the execution time of p_d by the execution time of p_r . We use the result as evaluation metrics for compiler autotuning techniques.

3.5 Procedure

For each program from PolyBench and cBench, we apply a compiler autotuning technique studied in this article and obtain its selected optimization combination within a given time limitation. In particular, for supervised learning-based techniques like BOCA and our proposed CompTuner, we first build a prediction model via training on some flag combinations. Due to the space limitations, we present the training cost of six programs in Table 2. The training cost includes the number of flag combinations (i.e., the number of training instances, given in Column "Number") and the corresponding time (given in Column "Time," in seconds) used by CompTuner to build a prediction model.

For different programs, the number of flag combinations and the time used by CompTuner to build a prediction model are different. From Table 2, we observe that on all six programs, the number of instances used to build the model is much smaller than 50. Note that CompTuner uses both 50 training instances and an accuracy of 0.96 as the termination criteria, and thus this observation indicates that our proposed CompTuner is able to build an accurate prediction model via fewer instances.

4 RESULTS AND ANALYSIS

4.1 Overall Effectiveness

4.1.1 Results of Different Techniques. To investigate the optimization effect of our proposed CompTuner on the target programs, we compare CompTuner with six state-of-the-art compiler

Table 3. Performance Improvement for Programs over -O3 on GCC

Technique	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup
CompTuner		1.077(3,107)		1.080(4,067)		1.042(2,573)		1.071(3,720)		1.091(3,127)
RIO		✗		✗		1.042(4,172)		✗		✗
GA		✗		✗		✗		✗		1.095(3,312)
TPE	P1	✗	P2	✗	P3	1.046(3,775)	P4	1.072(3,112)	P5	✗
BOCA		✗		✗		1.075(1,923)		✗		1.096(3,914)
OpenTuner		✗		✗		✗		1.075(4,691)		✗
COBYAN		✗		1.080(4,727)		1.068(1,092)		1.079(3,102)		✗
CompTuner		1.052(4,149)		1.073(5,549)		1.029(3,661)		1.025(2,976)		1.055(2,192)
RIO		✗		✗		1.033(3,264)		✗		✗
GA		1.058(3,671)		✗		✗		1.029(3,684)		✗
TPE	P6	✗	P7	✗	P8	✗	P9	1.031(2,637)	P10	✗
BOCA		✗		✗		1.037(4,082)		1.034(3,420)		1.055(3,026)
OpenTuner		✗		1.075(6,792)		1.039(4,970)		✗		✗
COBYAN		1.064(3,109)		✗		✗		1.028(4,116)		✗
CompTuner		1.267(4,174)		1.284(4,362)		1.327(3,907)		1.118(3,018)		1.036(5,527)
RIO		✗		✗		✗		1.119(3,976)		✗
GA		✗		✗		✗		✗		1.038(5,028)
TPE	C1	✗	C2	✗	C3	✗	C4	1.142(2,998)	C5	✗
BOCA		1.289(3,382)		✗		1.346(4,509)		1.127(3,192)		✗
OpenTuner		1.295(3,257)		1.319(4,074)		✗		✗		✗
COBYAN		✗		✗		✗		1.210(2,963)		1.052(5,874)
CompTuner		1.018(4,869)		1.038(4,892)		1.351(5,019)		1.020(3,628)		1.059(3,836)
RIO		✗		✗		✗		✗		1.059(4,458)
GA		1.018(3,960)		✗		✗		✗		✗
TPE	C6	✗	C7	✗	C8	1.352(5,927)	C9	1.023(3,785)	C10	✗
BOCA		1.025(4,027)		1.040(4,285)		✗		✗		1.062(2,705)
OpenTuner		✗		1.039(5,562)		✗		1.026(5,179)		✗
COBYAN		✗		✗		1.374(3,609)		1.022(3,212)		✗

autotuning techniques (i.e., RIO [13], GA [21], TPE [9], BOCA [12], OpenTuner [3], and COBYAN [7]). Tables 3 and 4 present the comparison results.

The state-of-the-art work BOCA [12] uses the iteration times as the termination condition (i.e., 60 iterations), but the iteration times does not match the tuning choice in practice. To simulate practical usage of compiler autotuning, we set the time limitation of CompTuner to be 6,000 seconds and run CompTuner within 6,000 seconds for each target program, recording (1) the runtime performance acceleration achieved by the generated flag combination (denoted as $performance_c$) and (2) how much time is used to generate this flag combination (denoted as $time_c$). The corresponding results are given by the *Speedup* values of CompTuner in the tables, where the values before the parentheses represent $performance_c$, and the values within the parentheses represent $time_c$. Note that $time_c$ is no larger than the given 6,000 seconds. To get $time_c$, we record how much time CompTuner consumes by the *time()* function of the Python library. If the time consumption reaches 6,000 seconds, we stop the tuning process. We record the optimal acceleration result and $time_c$ to achieve the result of this process. For the comparison technique, we repeat the tuning

Table 4. Performance Improvement for Programs over -O3 on LLVM

Technique	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup
CompTuner		1.042(2,052)		1.025(2,439)		1.041(2,182)		1.078(3,085)		1.066(2,994)
RIO		✗		1.027(2,271)		✗		✗		✗
GA		✗		✗		1.041(2,267)		✗		✗
TPE	P1	✗	P2	✗	P3	1.063(2,379)	P4	1.079(4,673)	P5	✗
BOCA		1.042(2,892)		1.030(2,189)		1.043(2,074)		1.078(3,562)		✗
OpenTuner		✗		1.025(2,174)		1.043(2,991)		✗		1.072(4,469)
COBYAN		✗		1.025(2,972)		✗		1.094(3,370)		✗
CompTuner		1.013(5,963)		1.023(5,728)		1.116(3,625)		1.014(2,230)		1.036(2,874)
RIO		1.015(6,124)		1.040(4,419)		✗		✗		✗
GA		✗		✗		✗		✗		1.036(3,627)
TPE	P6	✗	P7	✗	P8	✗	P9	✗	P10	1.038(3,135)
BOCA		1.015(5,732)		✗		1.124(3,824)		1.015(2,152)		1.039(2,610)
OpenTuner		✗		1.027(4,993)		1.143(4,179)		1.014(2,962)		✗
COBYAN		1.020(5,691)		✗		1.122(3,376)		✗		1.036(3,237)
CompTuner		1.030(3,327)		1.069(4,126)		1.204(3,294)		1.099(2,965)		1.019(5,524)
RIO		✗		✗		✗		✗		✗
GA		✗		1.069(4,362)		✗		1.099(3,642)		1.019(3,057)
TPE	C1	✗	C2	✗	C3	1.212(3,894)	C4	✗	C5	✗
BOCA		✗		1.072(3,920)		1.209(3,362)		✗		✗
OpenTuner		✗		✗		✗		1.108(2,711)		1.038(4,717)
COBYAN		1.030(3,914)		1.079(4,471)		✗		✗		✗
CompTuner		1.093(4,170)		1.025(2,749)		1.020(3,998)		1.036(3,557)		1.279(3,630)
RIO		✗		✗		1.023(4,872)		✗		✗
GA		✗		✗		✗		1.044(3,721)		✗
TPE	C6	✗	C7	1.025(3,889)	C8	✗	C9	1.038(3,659)	C10	✗
BOCA		1.093(5,598)		1.029(2,597)		✗		✗		✗
OpenTuner		✗		1.028(3,917)		1.022(4,537)		✗		1.294(3,127)
COBYAN		✗		1.035(4,130)		1.027(3,903)		1.040(4,144)		✗

process with a time limit of $1.5 * time_c$. Then we explore the performance of the compared techniques based on whether they can generate a compiler flag combination with the same runtime performance acceleration $performance_c$. In particular, we run each compared technique within a larger time limitation and record (1) whether any selected flag combination can reach the runtime performance acceleration, $performance_c$, and (2) how much time is used to select this flag combination. The corresponding results are given by the *Speedup* values of RIO, GA, TPE, BOCA, OpenTuner, and COBYAN in the tables, where ✗ represents that the compared technique does not select a flag combination with such high runtime performance acceleration and the values within the parentheses represent the time used to generate the corresponding combination. If the compared technique selects a flag combination with such high runtime performance acceleration, we present its runtime performance acceleration and the time. As each technique is repeated five times to alleviate the influence of randomness, we present the median results in these tables. The complete results are given in the reproducible package. Moreover, the smaller time in the table indicates that the corresponding autotuning technique is more effective. Note that for each target

program, we use a red box to emphasize the best result of all studied autotuning techniques, a blue box to emphasize the second-best result of all studied autotuning techniques, and bold font to emphasize the highest speedup of all studied autotuning techniques.

From Tables 3 and 4, the runtime performance acceleration of CompTuner on GCC ranges from 1.010 to 1.351, and on LLVM ranges from 1.013 to 1.279, indicating that (1) the flag combinations selected by CompTuner are always better than the default -O3 optimization given by GCC and LLVM, and (2) CompTuner always accelerates target programs through selected compiler flag combinations but achieves different accelerations on different target programs. Besides, the time of CompTuner used to select a flag combination with good runtime performance (i.e., the values within the parenthesis $time_c$) is usually smaller than the given time limitation of 6,000 seconds, indicating that CompTuner can be improved in the future.

Many results of the compared baselines (i.e., RIO, GA, TPE, BOCA, OpenTuner, and COBYAN) are marked with ✖, indicating that these compared baselines do not select a flag combination with the same runtime performance acceleration as CompTuner even if they are given with longer time limitations. Moreover, on one program (i.e., P1) on GCC, none of the baselines selects a flag combination with the same runtime performance acceleration as CompTuner given their longer time limitation. Among the seven baselines, BOCA performs much better, because RIO, GA, TPE, BOCA, OpenTuner, and COBYAN have 32, 29, 28, 17, 22, and 21 ✖s according to this table, respectively. For a compared technique, more ✖s represent that the technique cannot achieve the optimization results obtained by CompTuner even in more given time on more cases.

Furthermore, we also present the performance improvement over the compiler's default -O2 optimization in Tables 5 and 6. From these two tables, CompTuner achieves runtime performance acceleration over -O2 on GCC ranging from 1.002 to 2.349 and on LLVM ranging from 1.004 to 1.705, indicating that CompTuner selects better flag combinations than the compiler's default -O2 optimization. Moreover, among the compared techniques, CompTuner achieves the optimal results on six of seven programs on GCC/LLVM, indicating that CompTuner is a promising technique.

4.1.2 Statistics Analysis. To further investigate whether CompTuner outperforms the compared baselines, we statistically analyze the experimental results from two perspectives. First, we explore the best technique by analyzing in how many cases a technique achieves the best optimization result. Second, we perform a series of statistics analysis (including significant analysis) on CompTuner and the state-of-the-art technique BOCA.

Comparison among all techniques. From Tables 3 and 4, for each program, we identify the compiler autotuning technique that achieves the best and the second-best optimization results (which are addressed by the red and blue boxes). Then for each compiler autotuning technique we calculate the number of programs in which the corresponding technique achieves the best and the second-best optimization results, which are given by Table 7. In this table, the last three columns present the statistics results for GCC, LLVM, and both compilers.

From this table, among the 20 target programs, our proposed CompTuner achieves the best+the second-best optimization result on 6+8/7+9 programs of GCC/LLVM, respectively, while the compared RIO, GA, TPE, BOCA, OpenTuner, and COBYAN achieve the best results on only 1+0/2+0, 2+2/1+0, 1+2/0+1, 2+5/5+5, 2+1/3+3, and 6+2/3+2 programs of GCC/LLVM, respectively. This observation confirms that CompTuner outperforms the baselines in selecting a flag combination with good performance in most cases.

To sum up, our proposed technique CompTuner selects flag combinations better than the default -O3 optimization of GCC and LLVM. Moreover, in most cases it outperforms the compared compiler autotuning techniques, including the latest work, BOCA, demonstrating that CompTuner is promising.

Table 5. Performance Improvement for Programs over -O2 on GCC

Technique	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup
CompTuner		1.058(3,107)		1.088(4,067)		1.038(2,573)		1.040(3,720)		1.005(3,127)
RIO		✗		✗		1.041(4,172)		✗		✗
GA		✗		✗		✗		✗		1.006(3,312)
TPE	P1	✗	P2	✗	P3	1.042(3,775)	P4	1.045(3,112)	P5	✗
BOCA		✗		✗		1.071(1,923)		1.041(3,726)		1.007(3,914)
OpenTuner		✗		✗		✗		1.044(4,691)		✗
COBYAN		✗		1.088(4,727)		1.064(1,092)		1.049(3,102)		✗
CompTuner		1.017(4,149)		1.059(5,549)		1.036(3,661)		1.667(2,976)		1.401(2,192)
RIO		✗		✗		1.040(3,264)		✗		✗
GA		1.023(3,671)		✗		✗		1.673(3,684)		✗
TPE	P6	✗	P7	✗	P8	✗	P9	1.676(2,637)	P10	✗
BOCA		✗		✗		1.044(4,082)		1.681(3,420)		1.401(3,026)
OpenTuner		✗		1.061(6,792)		1.046(4,970)		✗		✗
COBYAN		1.029(3,109)		✗		✗		1.672(4,116)		✗
CompTuner		1.264(4,174)		1.274(4,362)		1.270(3,907)		1.124(3,018)		1.004(5,527)
RIO		✗		✗		✗		1.125(3,976)		✗
GA		✗		✗		✗		✗		1.006(5,028)
TPE	C1	✗	C2	✗	C3	✗	C4	1.148(2,998)	C5	✗
BOCA		1.286(3,382)		✗		1.288(4,509)		1.132(3,192)		✗
OpenTuner		1.292(3,257)		1.308(4,074)		✗		✗		✗
COBYAN		✗		✗		✗		1.216(2,963)		1.019(5,874)
CompTuner		1.319(4,869)		2.349(4,892)		1.069(5,019)		1.002(3,628)		1.011(3,836)
RIO		✗		✗		✗		✗		1.011(4,458)
GA		1.319(3,960)		✗		✗		✗		✗
TPE	C6	✗	C7	✗	C8	1.070(5,927)	C9	1.005(3,785)	C10	✗
BOCA		1.328(4,027)		2.353(4,285)		✗		✗		1.014(2,705)
OpenTuner		✗		2.351(5,562)		✗		1.008(5,179)		✗
COBYAN		✗		✗		1.087(3,609)		1.004(3,212)		✗

Comparison between CompTuner and BOCA. Since BOCA is the most recent work on compiler autotuning, we conduct a thorough comparison between CompTuner and BOCA.

From Tables 3 and 4, among the 20 target programs, our proposed CompTuner achieves better results than BOCA on 15/13 programs for compiler GCC/LLVM, respectively, while BOCA only achieves better results on 5/7 programs for compiler GCC/LLVM, respectively. Even when neither CompTuner nor BOCA achieves the best result, CompTuner often outperforms BOCA (e.g., P6 on GCC). That is, CompTuner outperforms BOCA in most programs. We further investigate the 28 programs (15 programs for GCC and 13 programs for LLVM) where CompTuner outperforms BOCA, comparing the time used by CompTuner and BOCA when achieving the same speedup.¹¹ On average, CompTuner saves 22.51% of time on the 15 programs for GCC and 26.65% of time on the 13 programs for LLVM. That is, on these programs, CompTuner uses much less time than

¹¹We set the time to 6,000 seconds if BOCA does not achieve the same optimization result in the fixed time.

Table 6. Performance Improvement for Programs over -O2 on LLVM

Technique	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup
CompTuner		1.042(2,052)		1.025(2,439)		1.041(2,182)		1.078(3,085)		1.066(2,994)
RIO		✗		1.027(2,271)		✗		✗		✗
GA		✗		✗		1.041(2,267)		✗		✗
TPE	P1	✗	P2	✗	P3	1.063(2,379)	P4	1.079(4,673)	P5	✗
BOCA		1.042(2,892)		1.030(2,189)		1.043(2,074)		1.078(3,562)		✗
OpenTuner		✗		1.025(2,174)		1.043(2,991)		✗		1.072(4,469)
COBYAN		✗		1.025(2,972)		✗		1.094(3,370)		✗
CompTuner		1.013(5,963)		1.023(5,728)		1.116(3,625)		1.014(2,230)		1.036(2,874)
RIO		1.015(6,124)		1.040(4,419)		✗		✗		✗
GA		✗		✗		✗		✗		1.036(3,627)
TPE	P6	✗	P7	✗	P8	✗	P9	✗	P10	1.038(3,135)
BOCA		1.015(5,732)		✗		1.124(3,824)		1.015(2,152)		1.039(2,610)
OpenTuner		✗		1.027(4,993)		1.143(4,179)		1.014(2,962)		✗
COBYAN		1.020(5,691)		✗		1.122(3,376)		✗		1.036(3,237)
CompTuner		1.030(3,327)		1.069(4,126)		1.204(3,294)		1.099(2,965)		1.019(5,524)
RIO		✗		✗		✗		✗		✗
GA		✗		1.069(4,362)		✗		1.099(3,642)		1.019(3,057)
TPE	C1	✗	C2	✗	C3	1.212(3,894)	C4	✗	C5	✗
BOCA		✗		1.072(3,920)		1.209(3,362)		✗		✗
OpenTuner		✗		✗		✗		1.108(2,711)		1.038(4,717)
COBYAN		1.030(3,914)		1.079(4,471)		✗		✗		✗
CompTuner		1.093(4,170)		1.025(2,749)		1.020(3,998)		1.036(3,557)		1.279(3,630)
RIO		✗		✗		1.023(4,872)		✗		✗
GA		✗		✗		✗		1.044(3,721)		✗
TPE	C6	✗	C7	1.025(3,889)	C8	✗	C9	1.038(3,659)	C10	✗
BOCA		1.093(5,598)		1.029(2,597)		✗		✗		✗
OpenTuner		✗		1.028(3,917)		1.022(4,537)		✗		1.294(3,127)
COBYAN		✗		1.035(4,130)		1.027(3,903)		1.040(4,144)		✗

Table 7. Number of Top-2 Performance Results

Technique	GCC	LLVM	All
CompTuner	6/8	7/9	13/17
RIO	1/0	1/0	2/0
GA	2/2	1/0	3/2
TPE	1/2	0/1	1/3
BOCA	2/5	5/5	7/10
OpenTuner	2/1	3/3	5/4
COBYAN	6/2	3/2	8/4

BOCA to achieve the same optimization results, which also indicates the outperformance of CompTuner over BOCA.

Furthermore, we perform significance analysis on the 40 cases in Tables 3 and 4¹² to compare the acceleration effect of CompTuner and BOCA. By performing Pair T-test on the time of the 40 cases, we get the p-value of 0.000115, which shows that the optimization flag settings generated by CompTuner are significantly better than those by BOCA.

We manually explore the optimization flag settings of some programs recommended by CompTuner and BOCA to learn why the former outperforms the latter. In particular, for each program we apply CompTuner and BOCA five times, manually analyze their recommended optimization flag settings, and find that the five optimization flag settings recommended by CompTuner are close to a large extent while the five optimization flag settings recommended by BOCA are very different. For example, for program P7 on GCC, CompTuner always enables a set of 30 optimization flags in all five experiments, while BOCA always enables only a set of 9 optimization flags. In other words, the flag combinations selected by BOCA for P7 in five experiments are very different. BOCA does not perform as well as CompTuner because its prediction model does not recommend impactful flags successfully.

4.1.3 Flag Combinations of CompTuner. To further investigate whether CompTuner selects suitable compiler optimization flags for a program, we conduct a small study on the flag combinations selected by CompTuner.

Flag combinations for P1 and C1. Here we take programs P1 and C1 as examples and manually analyze the flag combinations selected by CompTuner.¹³

For program P1, CompTuner selects 81 optimization flags, including 7 loop-related optimization flags, 6 branch-related optimization flags, and 68 flags for other program structures. In particular, we find that CompTuner selects many loop-related flags (e.g., -floop-unroll-and-jam, -floop-interchange). This result is consistent with our expectation, because the program P1 consists of five double-loop structures and one triple-loop structure, which accounts for most of the program. Among the remaining unselected flags, CompTuner does not select the -fcrossjumping flag, as the official GCC documentation interprets this flag as “The resulting code may or may not perform better than without cross-jumping.”

For program C1, CompTuner selects 85 optimization flags, including 3 loop-related flags, 9 branch-related flags, and 73 flags for other program structures. Compared with P1, C1 has more branch structures, so CompTuner selects the most branch-related flags for C1 (e.g., -fcse-follow-jumps, -fguess-branch-probability). Moreover, since C1 contains many calculations, CompTuner also selects the corresponding flags (e.g., -fipa-icf, -ftree-bit-ccp). Besides, CompTuner does not select the -fgcse flag for C1, as the official GCC documentation interprets this flag as “may get better run-time performance if adding -fno-gcse”.

Flag combination comparison with -O3. The compiler’s default -O3 optimization is extensively employed in both practical applications and research. To evaluate the speedup performance of our proposed CompTuner, it is essential to compare the flag combinations selected by CompTuner with the -O3 optimization. We select P1 on GCC as the examples and manually analyze the flag combinations selected by CompTuner. The analysis results are given in Table 8, where the second column presents the ratio of the runtime performance of the program compiled with the flag combinations selected by CompTuner to -O3 optimization. The third column lists the different flags between the

¹²In the same way as before, if BOCA fails to achieve the optimization result as CompTuner within the fixed time, its time is set to 6,000 seconds.

¹³We put the combinations selected by CompTuner for all programs on our website.

Table 8. Comparison of CompTuner and -O3 Optimization

Program	Performance	Different Flags	Same Flag Ratio
P1	1.077	-funroll-loops, -ftree-loop-vectorize...	0.86

Table 9. Compilers with Selected Optimization Flags

Compiler	Flags
GCC1	-fssa-phiopt, -fsched-interblock, -ftree-loop-distribution, -ftree-loop-vectorize, -fcse-follow-jumps, -fthread-jumps, -finline-small-functions, -fipa-cp, -ftree-dce, -ftree-dse
GCC2	-fdelayed-branch, -fdse, -fipa-reference, -fgcse, -ftree-sink, -fipa-rrp, -fipa-bit-cp, -floop-unroll-and-jam, -floop-interchange, -fpeel-loops
LLVM1	-targetlibinfo, -tbaa, -basicaa, -globalopt, -deadargelim, -ipsccp, -inline, -instcombine, -basiccg, -prune-eh
LLVM2	-functionattrs, -argpromotion, -domtree, -tailcallelim, -early-cse, -lazy-value-info, -jump-threading, -correlated-propagation, -simplifycfg, -instcombine

default -O3 optimization and the flags selected by CompTuner. The last column presents the ratio of the same optimization flags selected by both CompTuner and -O3 optimization.

From Table 8, on P1, the acceleration ratio achieved by CompTuner is 1.077, indicating its superiority over -O3 optimization. From the last column, the value of “Same Flag ratio” on P1 is 0.86, indicating that CompTuner selects nearly 20 flags different from -O3 optimization.

From the third column, the flags selected by -O3 optimization are different from CompTuner to some extent, and such difference leads to the varied performance of P1. For example, the -O3 optimization enables optimization flags like “-funroll-loops” and “-ftree-loop-vectorize,” while CompTuner does not for P1. Through further manual analysis, we find that enabling these two flags together seems to decrease the program runtime performance of P1. Besides, as the -O3 optimization may not be the best optimization configuration, the same flag ratio between CompTuner and the -O3 optimization (given by the last column) does not necessarily correlate with runtime performance (given by the second column).

Flag combination comparison with the best optimization configuration. It is also interesting to learn how good the flag combinations selected by CompTuner are by comparing against the best flag combinations (i.e., best optimization configurations), and thus we conduct a small study in this section. We can hardly conduct a study on the whole set of optimization flags because a GCC compiler contains more than 100 optimization flags.¹⁴ Alternatively, we select the first three programs of Polybench and Cbench, i.e., programs P1, P2, P3, C1, C2, C3, and construct compilers with a smaller number of optimization flags. That is, for each studied compiler (i.e., GCC and LLVM), we construct two sets of compiler optimization flags, each of which contains 10 randomly selected compiler optimization flags. The four sets of compiler optimization flags are shown in Table 9. Within a set of 10 optimization flags we find its best configuration through a search process and compare the best configuration against flag combinations selected by CompTuner.

Due to space limitations, we do not present the specific flag combinations selected by CompTuner and the best configurations in the article, but they are on the website. Alternatively, we analyze the difference between their selected flag combinations and the difference in runtime performance acceleration. The analysis results are given by Table 10, where the second column presents the ratio of the runtime performance of the program compiled with the flag combinations selected

¹⁴The searching space is larger than 2^{100} , and thus it is very costly to find the best optimization configuration for even one program.

Table 10. Comparison of CompTuner and Best Configurations

Program	Performance	Unnecessary Flags	Missing Flags	Accuracy
GCC1				
P1	0.959	-fssa-phiopt	-fthread-jumps	0.8
P2	0.942	-fthread-jumps	-finline-small-functions, -ftree-dse	0.7
P3	0.964	-	-fsched-interblock, -ftree-loop-vectorize	0.8
C1	0.981	-ftree-dce	-	0.9
C2	0.913	-ftree-loop-vectorize	-finline-small-functions, -fipa-cp	0.7
C3	0.954	-ftree-dse	-fcse-follow-jumps	0.8
GCC2				
P1	0.966	-	-ftree-sink	0.9
P2	0.953	-fgcse	-floop-interchange	0.8
P3	0.927	-fipa-vrp	-fipa-reference, -fgcse	0.7
C1	0.938	-fipa-bit-cp	-fgcse	0.8
C2	0.939	-fipa-bit-cp	-fipa-vrp	0.8
C3	0.974	-ftree-sink, -floop-unroll-and-jam	-	0.8
LLVM1				
P1	1.000	-	-	1.0
P2	0.976	-tbaa	-	0.9
P3	0.955	-instcombine	-ipsccp	0.8
C1	0.982	-	-globalopt	0.9
C2	0.977	-	-basiccg, -prune-eh	0.8
C3	0.941	-deadargelim	-basiccg	0.8
LLVM2				
P1	0.984	-early-cse	-	0.9
P2	0.962	-domtree	-jump-threading	0.8
P3	0.979	-tailcallelim	-simplifcfg	0.8
C1	0.991	-	-functionattrs	0.9
C2	0.946	-domtree	-jump-threading, -simplifcfg	0.7
C3	0.959	-argpromotion	-correlated-propagation	0.8

by CompTuner to the best configurations, the third column presents the flags selected by CompTuner, while not selected by the best configurations, the fourth column presents the flags selected by the best configurations, while not selected by CompTuner, and the last column presents the ratio of optimization flags whose settings are the same for CompTuner and the best configurations.

From the second column, the performance of CompTuner is usually close to 1, indicating that the optimization flags selected by CompTuner achieve runtime performance acceleration close to the best configurations. From the last column, the accuracy is always larger than 0.7 and mostly larger than 0.8, indicating that the optimization configurations recommended by CompTuner are very close to the best configurations. This conclusion can also be drawn by the third and fourth columns; i.e., CompTuner selects only at most two unnecessary flags or misses at most two flags.

Besides, we also observe that the runtime performance acceleration of CompTuner (i.e., given by the second column) usually increases along with its accuracy (i.e., given by the last column). This observation is as expected because flag combinations with high accuracy tend to have much overlap with the best configurations.

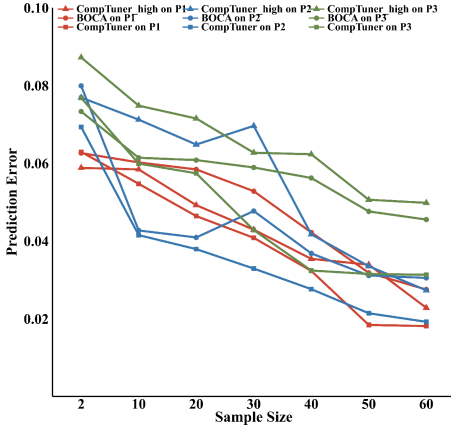


Fig. 2. Prediction error rate of PolyBench.

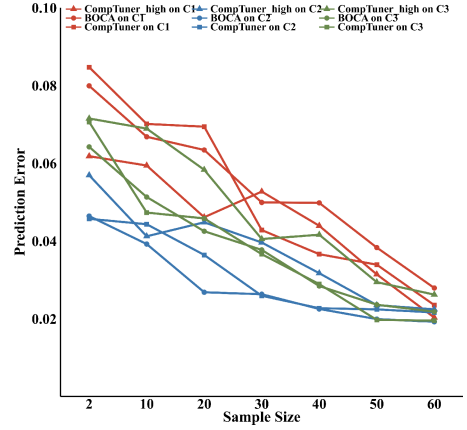


Fig. 3. Prediction error rate of cBench.

4.2 Ablation Analysis

In this section, we perform an ablation analysis to investigate the contribution of two components of CompTuner, which are a prediction model (given by Section 2.2) and intelligent searching process (given by Section 2.3).

4.2.1 Prediction Model Analysis. To investigate whether the proposed lightweight learning approach in CompTuner is well designed, similar to BOCA, we build a prediction model by using only flag combinations with good runtime performance, resulting in a variant of CompTuner (denoted as *CompTuner_{high}*). Besides, BOCA is the state-of-the-art supervised learning-based technique that also builds a prediction model. However, different from CompTuner, BOCA builds a prediction model by selecting the flag combinations with the highest *expected improvement*, and thus in this study we also compare against the prediction model built by BOCA. To sum up, in this study we compare the prediction models of CompTuner, *CompTuner_{high}*, and BOCA in terms of their prediction accuracy and runtime performance acceleration. Besides, all these approaches start building a prediction model with some randomly selected flag combinations. To alleviate the influence of this random selection, we use the same randomly selected flag combinations as the input of the three approaches and observe the prediction accuracy of each compared technique by gradually selecting more flag combinations in building the prediction model.

For each target program we randomly generate 100 compiler flag combinations and record their actual runtime performance as the ground truth. For each compared technique (i.e., CompTuner, *CompTuner_{high}*, and BOCA), we first construct prediction models with various numbers of training data (i.e., the number is set to be 2, 10, 20, 30, 40, 50, and 60) and use these prediction models to predict the runtime performance of the randomly selected flag combinations, recording the accuracy of each prediction model. The results are given in Figures 2 and 3, where the horizontal axis represents the size of the training set, and the vertical axis represents the prediction error rate, i.e., the absolute difference between predicted and actual runtime performance divided by the actual runtime performance. A small prediction error rate means a good prediction model. In particular, this ablation study is conducted on GCC by using the first three target programs from PolyBench and cBench as the representatives, which are P1, P2, P3, and C1, C2, C3. To distinguish these programs, in the figure we use red to denote P1 and C1, blue to denote P2 and C2, and green to denote P3 and C3.

From these figures, although these techniques take the same flag combinations as input to build a prediction model, their corresponding models have different prediction error rates. The

Table 11. Results of the Ablation Study

Technique	P1	P2	P3	C1	C2	C3
CompTuner	1.058	1.063	1.034	1.204	1.283	1.298
<i>CompTuner_{high}</i>	1.042	1.044	1.029	1.217	1.241	1.256
<i>CompTuner_{impact}</i>	1.054	1.069	1.029	1.179	1.299	1.231

prediction error rate of CompTuner is smaller than that of *CompTuner_{high}* and BOCA in most programs (i.e., four of the six programs when the input size is no smaller than 20). In particular, the prediction error rate of CompTuner on P3 is only 0.03, while those of *CompTuner_{high}* and BOCA are larger than 0.05. Although sometimes (e.g., in P1) the prediction error rate of CompTuner is slightly larger than *CompTuner_{high}* or BOCA at the very beginning of model construction (i.e., fed with a very small number of flag combinations), the former becomes close to the latter when 20 flag combinations are used. Moreover, the prediction error rate of CompTuner is usually smaller than *CompTuner_{high}* and BOCA when more than 40 input data are used, which demonstrates that CompTuner can always produce a prediction model with high accuracy given a reasonable number of training data. Moreover, compared with *CompTuner_{high}*, the curves of CompTuner are close, indicating that the prediction accuracy of its models on various target programs is stable.

Then we compare the runtime performance acceleration of CompTuner and *CompTuner_{high}* (whose only difference lies in how to build a prediction model) following the setup of Section 4.1. Note that we do not compare the runtime performance acceleration of CompTuner and BOCA because we have done so in Section 4.1. The comparison results are given by Row CompTuner as well as Row *CompTuner_{high}* in Table 11. Compared to *CompTuner_{high}*, CompTuner achieves better optimization results on five of the six target programs. These comparative results reveal that the way of constructing the model by CompTuner not only constructs a more accurate prediction model but also produces a better acceleration effect on the programs.

4.2.2 Intelligent Searching Analysis. To investigate whether the improved particle swarm optimization algorithm in CompTuner is well designed, we replace this optimization selection process with that of BOCA, which generates several compiler flag combinations using all combinations of impactful compiler optimization flags and some combinations of less impactful compiler optimization flags, resulting in a variant of CompTuner (denoted as *CompTuner_{impact}*). The comparison results are also given in Table 11.

From this table, CompTuner achieves better optimization results on most programs (i.e., four of the six target programs), confirming the effectiveness of intelligent searching in CompTuner. We also note that *CompTuner_{impact}* achieves better optimization results than CompTuner on program P2, the smallest program of the six target programs, and thus we hypothesize that the performance of CompTuner is more obvious on larger programs. That is, combining impactful flags with our CompTuner may be a promising direction in the future.

To verify our hypothesis, we choose some smaller programs (i.e., P1, P2, and P3) and larger programs (i.e., C4 and C5) and compare their performance of CompTuner and *CompTuner_{impact}*, as shown in Table 12. From this table, the advantage of CompTuner over *CompTuner_{impact}* is more obvious on larger programs. In particular, P1, P2, and P3 represent programs with 200+ lines, while C4 and C5 represent programs with 2,000+ lines. According to the experimental results, on the selected five programs, *CompTuner_{impact}* outperforms CompTuner only on the small program P2 (with 218 lines). Moreover, CompTuner performs more better on the large program C5 (with 22,321 lines).

Table 12. Comparison of CompTuner and *CompTuner_{impact}*

Technique	P1	P2	P3	C4	C5
CompTuner	1.058	1.063	1.034	1.114	1.034
<i>CompTuner_{impact}</i>	1.054	1.069	1.029	1.102	1.021
Difference	0.004	-0.006	0.005	0.012	0.013

5 THREATS TO VALIDITY

The internal threat mainly comes from the implementation of compiler autotuning approaches. To reduce this threat, we use the reproducible package of the compared approaches and re-implement the compared approaches strictly following their papers if their reproducible package is not available. Besides, the authors review the implementation to further reduce this threat.

The external threat mainly comes from the target programs and compilers. To reduce this threat, we use the same benchmarks and popular compilers as previous work did [12].

The construct threat lies in the metrics used in the evaluation. GCC and LLVM have several recommended optimization settings such as -O1, -O2, -O3, and -Ofast. -Ofast is reported to be an addition to -O3 with some unconventional optimizations that are achieved by breaking some international standards (e.g., *-fno-trapping-math* disables program aborts caused by floating-point exceptions, which may lead to infinite loops or other strange behavior if there is an incorrect calculation). -O1 does not enable many optimization flags. Therefore, in the literature -Ofast is not generally recommended and the existing work on compiler tuning is usually compared against -O3. As prior work did [12], we also use the runtime performance acceleration over the compiler's default -O3 optimization and further reduce this threat by also including the comparison against -O2 optimization.

6 DISCUSSION

In this section, we conduct five small studies to investigate the influence of compiler autotuning time, compiler versions, the number of optimization flags, multiple-phase model construction, and phase ordering technique.

6.1 Influence of the Tuning Time

Compiler autotuning time may influence the performance of autotuning techniques, and thus we compare the performance of compiler autotuning techniques by changing their tuning time limitation. In particular, we repeat the experiment in Section 4.1 on only six programs (i.e., P1, P2, P3, C1, C2, and C3) by setting the time limitation to be 2,000 seconds, while the time limitation used in Section 4.1 is 6,000 seconds. The results are given in Table 13.

From this table, CompTuner outperforms the compared techniques in most programs within the 2,000-second evaluation. In particular, CompTuner achieves the highest speedup on 2/1 programs (addressed in bold font) of GCC/LLVM, respectively, while the compared RIO, GA, TPE, BOCA, OpenTuner, and COBYAN achieve the highest speedup on only 0/0, 0/1, 0/1, 2/1, 1/1, and 1/1 programs of GCC/LLVM. Compared with results within 6,000 seconds (given by Tables 3 and 4), the performance improvement of these autotuning techniques increases from 2,000 seconds to 6,000 seconds. This observation is as expected because longer tuning time may increase the model's performance. However, the advantage of CompTuner over the compared techniques holds from 2,000 seconds to 6,000 seconds.

In order to observe the performance of the different techniques over a longer timeout, we perform the same experiment over 10,000 seconds on the six programs, but due to time constraints we only repeat it once. The results are given in Table 14.

Table 13. Performance Improvement for Programs over -O3 on GCC and LLVM in 2,000 Seconds

Technique	P1	P2	P3	C1	C2	C3
GCC						
CompTuner	1.051(1,923)	1.047(1,662)	1.041(1,825)	1.073(1,302)	1.105(1,467)	1.135(1,481)
RIO	✗	✗	✗	1.081(1,119)	✗	✗
GA	✗	✗	1.041(1,858)	✗	✗	✗
TPE	✗	✗	✗	✗	1.108(1,549)	✗
BOCA	✗	✗	1.046(1,532)	✗	✗	1.169(1,326)
OpenTuner	✗	1.047(1,748)	✗	✗	1.116(1,283)	✗
COBYAN	✗	✗	✗	1.106(1,748)	✗	1.144(1,422)
LLVM						
CompTuner	1.038(1,725)	1.022(1,554)	1.037(1,475)	1.020(1,632)	1.046(1,913)	1.103(1,844)
RIO	✗	✗	1.038(1,926)	✗	✗	✗
GA	✗	1.024(1,824)	✗	1.022(1,547)	✗	✗
TPE	✗	1.040(1,998)	1.037(1,216)	✗	✗	✗
BOCA	✗	1.023(1,794)	✗	✗	1.052(1,565)	✗
OpenTuner	1.039(1,788)	✗	✗	✗	1.046(1,924)	✗
COBYAN	✗	1.028(1,740)	1.044(1,979)	1.020(1,380)	✗	✗

Table 14. Performance Improvement for Programs over -O3 on GCC and LLVM in 10,000 Seconds

Technique	P1	P2	P3	C1	C2	C3
GCC						
CompTuner	1.109(7,694)	1.097(8,545)	1.083(7,920)	1.337(7,268)	1.382(7,159)	1.364(8,623)
RIO	✗	✗	✗	✗	✗	✗
GA	✗	✗	✗	✗	✗	1.366(8,229)
TPE	✗	✗	1.083(8,692)	✗	✗	✗
BOCA	✗	✗	1.097(7,735)	✗	✗	✗
OpenTuner	✗	1.116(8,929)	✗	1.352(6,915)	✗	✗
COBYAN	✗	✗	1.094(8,136)	✗	✗	1.379(7,672)
LLVM						
CompTuner	1.084(9,032)	1.079(8,717)	1.123(6,832)	1.094(8,612)	1.110(8,235)	1.210(7,392)
RIO	✗	1.079(9,554)	✗	✗	✗	✗
GA	✗	✗	✗	1.094(9,753)	✗	✗
TPE	✗	1.085(9,502)	✗	✗	✗	✗
BOCA	✗	1.094(8,337)	✗	✗	1.127(8,992)	✗
OpenTuner	✗	✗	✗	✗	1.119(9,827)	✗
COBYAN	✗	1.088(9,331)	✗	1.107(7,463)	1.135(7,604)	✗

Table 15. Comparison of CompTuner and BOCA on GCC 12.2.0 and LLVM 15.0.0

Technique	P1	P2	P3	C1	C2	C3
GCC 12.2.0						
CompTuner	1.014(4,517)	1.028(5,262)	1.008(3,884)	1.122(5,753)	1.095(4,031)	1.169(3,983)
BOCA	✗	✗	1.014(5,105)	1.122(5,964)	1.174(3,372)	✗
LLVM 15.0.0						
CompTuner	1.021(4,362)	1.024(3,729)	1.032(3,380)	1.025(4,171)	1.041(5,514)	1.096(4,793)
BOCA	✗	1.038(4,413)	✗	1.031(3,582)	✗	1.114(4,623)

From this table, CompTuner outperforms the compared techniques in most programs within the 10,000-second evaluation. In particular, CompTuner achieves the highest speedup (addressed in bold font) on 2/3 programs of GCC/LLVM, respectively, while the compared RIO, GA, TPE, BOCA, OpenTuner, and COBYAN achieve the highest speedup on only 0/0, 0/0, 0/0, 1/1, 2/0, and 1/2 programs of GCC/LLVM. Compared with results within 2,000 seconds and 6,000 seconds (given by Tables 3, 4, and 13), within 10,000 seconds, CompTuner achieves better performance over these compared techniques.

6.2 Influence of the Compiler Version

To reduce the threat to validity resulting from compiler versions, we further evaluate CompTuner by comparing against BOCA [12] on the latest versions of GCC and LLVM, i.e., GCC 12.2.0 and LLVM 15.0.0, using the same programs as Section 6.1. The results are given in Table 15.

From this table, all the data of CompTuner are larger than 1, indicating that our proposed CompTuner achieves better results than -O3 optimization on the latest versions of GCC and LLVM. CompTuner achieves the best optimization result on 5/4 programs of GCC 12.2.0/LLVM 15.0.0, respectively, while the compared BOCA achieves the best results on only 1/2 programs of GCC 12.2.0/LLVM 15.0.0. Note that for P3 on GCC and P2 on LLVM, we mark the results of CompTuner as the best result (addressed by red box) because BOCA spends more time to achieve the same runtime performance acceleration as CompTuner. In other words, BOCA cannot select a flag combination in GCC with the acceleration 1.008 within 3,884 seconds on P3, nor in LLVM with 1.024 within 3,729 seconds on P2.

To sum up, on the latest version of GCC and LLVM, our proposed CompTuner still outperforms the state-of-the-art BOCA on most programs. Moreover, combined with the observations on old versions of GCC and LLVM (i.e., results from Tables 3 and 4), CompTuner performs much better than BOCA on the latest version compared with the old version of the compilers.

6.3 Influence of the Number of Optimization Flags

From the experimental results in Section 4.1, we find that the speedup of these techniques on GCC 8.3.0 is better than GCC 12.2.0 and LLVM 9.0.0 in general. As the latter two compilers have more optimization flags than GCC 8.3.0, we suspect that the number of optimization flags may have affected the performance of the compiler autotuning technique.

To verify this hypothesis, we construct GCC 12- and LLVM 9- by reducing the number of the optimization flags of GCC 12.2.0 and LLVM 9.0.0 to the same number of optimization flags of GCC

Table 16. Comparison of CompTuner on GCC 8.3.0, GCC 12-, and LLVM 9-

Technique	P1	P2	P3	C1	C2	C3
GCC 8.3.0	1.077(3,107)	1.080(4,067)	1.042(2,573)	1.267(4,174)	1.274(4,362)	1.327(3,907)
GCC 12-	1.036(3,234)	1.054(4,971)	1.011(3,914)	1.175(4,584)	1.131(3,470)	1.192(3,285)
LLVM 9-	1.049(3,052)	1.086(3,549)	1.057(2,914)	1.072(3,972)	1.094(5,628)	1.182(3,014)

Table 17. Comparison of CompTuner and *CompTuner_{one-time}*

Technique	P1	P2	P3	C1	C2	C3
Error rate						
CompTuner	0.024	0.026	0.027	0.028	0.022	0.028
<i>CompTuner_{one-time}</i>	0.028	0.021	0.031	0.034	0.018	0.030
Speedup						
Technique	P1	P2	P3	C1	C2	C3
CompTuner	1.065	1.053	1.049	1.288	1.202	1.259
<i>CompTuner_{one-time}</i>	1.043	1.058	1.028	1.252	1.174	1.270

8.3.0.¹⁵ In particular, we repeat the experiment in Section 4.1 by applying CompTuner to the six programs (i.e., P1, P2, P3, C1, C2, and C3). The results are given in Table 16.

From Table 16, CompTuner achieves higher speedup on 4/6 programs (addressed in bold font), which illustrates that the default -O3 optimization of GCC 12- and LLVM 9- has a better optimization effect, despite the removal of some of the optimization flags from the origin compilers. However, combining the results from Tables 15 and 16, CompTuner performs better on GCC 12- and LLVM 9- compared with GCC 12.2.0 and LLVM 9.0.0, indicating that the reduction of the compiler optimization flags facilitates CompTuner to perform a more comprehensive search.

6.4 Influence of Multiple-phase Model Construction

Our approach CompTuner builds a prediction model through multiple-phase learning instead of single-phase learning, with the purpose of improved prediction accuracy. To verify whether multiple-phase model construction performs better than single-phase learning, we implement a variant of CompTuner by replacing its multiple-phase learning by one-phase learning, which is called *CompTuner_{one-time}*, and compare the performance of CompTuner and *CompTuner_{one-time}* on the same six programs as Section 6.1. For fair comparison, *CompTuner_{one-time}* uses 50 flag combinations randomly selected from candidate combinations to build the prediction model, while CompTuner uses 50 other flag combinations selected by multi-phase selection to build the model. The results are given in Table 17.

To verify the effectiveness of the performance-prediction model-building part of our technique, we compare the performance of our technique with *CompTuner_{one-time}*, which constructs the performance-prediction model by one-phase selection from candidate combinations.¹⁶ We choose six programs on GCC to compare the two techniques.

From Table 17, CompTuner outperforms *CompTuner_{one-time}* on 4/6 programs and constructs a more accurate prediction model on 4/6 programs. Moreover, a prediction model with low error rate usually results in flag combinations with good runtime performance. However, on program

¹⁵For a fair comparison, in the default -O3 optimization of GCC 12- and LLVM 9-, we only enable the optimization flags that are enabled in the default -O3 optimization of GCC 12.2.0 and LLVM 9.0.0 and within the reduced optimization flag set of GCC 12- and LLVM 9-.

¹⁶We all select 50 samples for the two performance-prediction model construction ways.

Table 18. Comparison of CompTuner and AutoPhase on LLVM

Technique	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup
CompTuner	P1	1.042(2,052)	P2	1.025(2,439)	P3	1.041(2,182)	P4	1.078(3,085)	P5	1.066(2,994)
AutoPhase		1.029(3,742)		1.037(3,801)		1.068(3,862)		1.056(5,731)		-/-
CompTuner	P6	1.013(5,963)	P7	1.023(5,728)	P8	1.116(3,625)	P9	1.014(2,230)	P10	1.036(2,874)
AutoPhase		-/-		-/-		-/-		-/-		1.026(4,462)
CompTuner	C1	1.030(3,327)	C2	1.069(4,126)	C3	1.204(3,294)	C4	1.099(2,965)	C5	1.019(5,524)
AutoPhase		-/-		1.084(5,982)		1.189(5,731)		1.093(5,102)		-/-
CompTuner	C6	1.093(4,170)	C7	1.025(2,749)	C8	1.020(3,998)	C9	1.036(3,557)	C10	1.279(3,630)
AutoPhase		-/-		-/-		-/-		1.033(2,492)		-/-

C2, *CompTuner_{one-time}* constructs a prediction model with high accuracy, while its runtime performance is lower than CompTuner. We suspect the reason to be that CompTuner chooses a better starting point for the improved particle swarm process.

6.5 Comparison against Phase Ordering

As mentioned in Section 1, flag selection and phase ordering compose a compiler optimization setting, where the former targets selecting a set of optimization flags, whereas the latter targets deciding the order in which these optimization flags are applied. The proposed approach CompTuner targets the former, but it is still interesting to learn how the proposed approach performs compared to techniques of the latter. In this exploration study, we include AutoPhase [27], a state-of-the-art deep reinforcement learning-based technique for phase ordering, and compare it with our proposed CompTuner on the 20 programs from cBench and PolyBench. As these two compared approaches target two different problems (i.e., phase ordering and flag selection) but with the same goal (i.e., enhancing runtime performance of a compiled program), we try to pose a fair comparison based on the following procedure. First, we apply each approach within the same time budget (i.e., still 6,000 seconds). Then we feed their outputs (i.e., applying order of flags and selected flag combination) to the compiler LLVM 9.0.0 and record the corresponding runtime performance of compiled programs. The results are shown in Table 18. In particular, if AutoPhase fails to complete the optimization within the limited time (i.e., 6,000 seconds), we represent the speedup result as “-/-”.

From Table 18, AutoPhase cannot finish phase ordering within the given time on 11/20 programs. On the remaining 9 programs, CompTuner outperforms AutoPhase on 6 programs, indicating that CompTuner is promising compared to a phase ordering approach.

7 RELATED WORK

This article targets compiler autotuning and thus is most related to compiler autotuning. Besides, this article is also related to software tuning in general. Therefore, in this section, we will first give a quick review on the existing work on software tuning in Section 7.1 and then compiler autotuning in Section 7.2.

7.1 Software Tuning

Tuning is an important problem in both engineering and research. Many systems and models often are set with a very large number of configurable parameters, which can be tuned to satisfy different demands. For example, data management systems like MySQL have parameters whose values can be tuned to improve performance and ensure security, web service systems like Nginx have parameters whose values can be tuned to ensure their performance and stability [24],

embedded systems have parameters whose value combination can be tuned to consume less energy at runtime [52], deep learning models have a lot of hyperparameters that can be tuned to make the models obtain better performance (e.g., high prediction accuracy) [34, 44], and some software systems have many parameters whose values can be tuned to achieve better runtime performance in different hardware environments [54].

Tuning occurs in various software and systems, but their common difficulty lies in how to efficiently explore the huge parameter space. To address this problem, researchers proposed many tuning techniques, which can be classified into search-based techniques and learning-based techniques. In particular, besides random-based techniques [13] and genetic algorithm-based techniques [21, 26], simulated annealing algorithm-based techniques [19] were widely used, which found an optimal parameter setting through the actual execution of the target system and a specific search strategy. However, due to software complexity, actual execution consumes a lot of time and computing resources, resulting in low optimization efficiency. The learning-based techniques build prediction models through some data and then predict the performance of parameter combinations instead of actual execution. To find better parameter combinations quickly, many sampling techniques were proposed to approximate the optimal parameter combinations in the large parameter space. For example, Henard et al. [25] proposed a multi-objective optimization technique to reduce the search space by constraints, Nair et al. [33] and Victoria and Maragatham [53] proposed Bayesian optimization-based techniques to produce optimal samples through evaluating the performance of the samples by acquisition functions, Dogru et al. [15] proposed a reinforcement learning-based technique that uses a reward function to obtain the future benefits of the sampling to gradually select the parameter combinations in the search space, and Cummins et al. [14] proposed a deep neural network that learns heuristics over raw code, constructs appropriate representations of the code, and learns how best to optimize.

7.2 Compiler Autotuning

Compiler autotuning aims at recommending optimization settings to improve program runtime performance. Typically, compiler autotuning consists of flag selection and phase ordering. Phase ordering aims at deciding the order in which the selected optimization flags are applied. For example, Ashouri et al. [5] proposed a learning-based technique that clusters the optimization passes of LLVM into different clusters to predict the speedup of a complete combination by a pre-train prediction model. Huang et al. [27] proposed a reinforcement learning-based technique that predicts the best next flag by features of the program and previously applied passes. As this article focuses on flag selection instead of phase ordering, we classify the existing techniques on flag selection [4, 6] into unsupervised learning-based methods and supervised learning-based methods and review the existing techniques following this classification to review.

Unsupervised learning-based techniques. The unsupervised learning-based techniques start the optimization process from randomly generated samples and use some searching strategies to select new flag combinations. Chen et al. [13] proposed a random iterative optimization technique that aims to select a desired flag combination by generating new combinations in a random way and is usually regarded as the baseline for compiler autotuning techniques. To trace the searching process, some techniques (e.g., hill-climbing-based techniques [2] and genetic algorithm-based techniques [21, 26, 35, 43]) were proposed to guide compiler autotuning through evolution. In addition, Pérez Cáceres et al. [39] proposed Irace to generate new compiler combinations based on performance distribution. The unsupervised learning-based techniques search and obtain a desired flag combination by actually compiling and executing the target program with a large number of selected flag combinations, which is very time-consuming.

Supervised learning-based techniques. The supervised learning-based techniques construct a prediction model by flag combinations (with the corresponding runtime performance of the compiled program) and then explore the optimization space by some predefined heuristics. In particular, Agakov et al. [1] randomly selected a number of flag combinations to build a prediction model and generated new flag combinations with Markovian. Later, Ashouri et al. [8] proposed a Bayesian network-based technique that first builds a prediction model and then generates new combinations through a Bayesian process that gradually approximates the desired solution. Haj-Ali et al. [23] proposed to extract the loop structures of the program, embed these loops, and then dynamically determine the flag combination for all the loops through deep reinforcement learning.

Recently, Chen et al. [12] proposed another Bayesian network-based technique, BOCA, that first builds a prediction model to distinguish impactful flags from less impactful flags and then generates new combinations through a Bayesian process.

Our proposed CompTuner is also a supervised learning-based technique, but it is different from the existing supervised learning-based techniques in two aspects. First, CompTuner builds a prediction model via multiple phases of learning so as to alleviate the existing cost issue. Second, CompTuner selects flag combinations through an improved particle swarm optimization algorithm, which alleviates the performance issue by considering both global and local optima.

Moreover, the supervised learning-based techniques usually use Gaussian models as prediction models. Due to the excessive number of compiler compilation flags, the models constructed by such techniques usually suffer from accuracy issues, which are addressed by the improved particle swarm optimization algorithm of our CompTuner.

The most relevant work is BOCA [12]. In particular, BOCA first iteratively selects combinations to construct a prediction model, which is used to predict whether an optimization flag is impactful, then explores the search space for impactful flags and space for less impactful flags separately. CompTuner is also a supervised learning-based technique, and it is similar to BOCA because both of them build a prediction model iteratively with a small number of data. However, the purpose of their prediction models is different. The model of BOCA is mainly to identify whether an optimization flag is impactful, whereas the model of CompTuner is to predict the runtime performance of a flag combination. Moreover, when building a prediction model, BOCA uses only the combination with good runtime performance, whereas our CompTuner uses combinations with diversity.

8 CONCLUSION

To alleviate human efforts on compiler tuning for a specific program, several compiler autotuning techniques have been proposed in the literature, but all suffer from cost and performance issues. In this article, we propose CompTuner, a multiple-phase learning-based compiler autotuning technique. To alleviate the cost issue, CompTuner iteratively builds a prediction model with a very small number of data to predict the runtime performance of a flag combination instead of actual execution. To alleviate the performance issue, CompTuner designs an improved particle swarm optimization algorithm, which balances the local optimum and global optimum through modifying acceleration constants. According to the experimental results on GCC and LLVM, the proposed CompTuner achieves the best runtime performances in most target programs, significantly outperforming the state-of-the-art compiler autotuning techniques. Moreover, the ablation analysis also demonstrates the contribution of each component in CompTuner.

APPENDIX

In this section, we present another experiment on the remaining 20 programs of PolyBench and the remaining 18 programs of cBench. Different from Section 4, in this experiment, we apply each

compiler autotuning technique to each program only once. On these programs, we obtain consistent conclusions as the previous sections.

In detail, we present the information on these programs, the results of each technique on different programs, and a comparison of the statistical results for each technique.

Program information. Table 19 presents the statistics of these compiled programs. The first 20 programs are from PolyBench and the others are from cBench. That is, in this experiment, we have 38 compiled programs that cover a wide range of practical functions, ranging from 170 to 26,000 lines of code.

Results of different techniques. The experimental results are given in Tables 20–23. Similar to Section 4.1.1, in these tables, for each target program, we use a red box to emphasize the best result of all studied autotuning techniques, a blue box to emphasize the second-best result of all studied autotuning techniques, and a bold font to emphasize the highest speedup of all studied autotuning techniques.

From these tables, the runtime performance acceleration of CompTuner on GCC ranges from 1.029 to 1.255 and on LLVM ranges from 1.028 to 1.232, indicating that the flag combinations selected by CompTuner are always better than the default -O3 optimization given by GCC and LLVM on other programs.

Statistics analysis. To further compare the performance of CompTuner and the other techniques, we perform statistics on these tables and present the results in Table 24. From this table, among the 38 target programs, our proposed CompTuner achieves the best+the second-best optimization result on 11+15/10+13 programs of GCC/LLVM, respectively, while the compared RIO, GA, TPE, BOCA, OpenTuner, and COBYAN achieve the best and second-best results on only 3+1/0+3, 3+1/2+2, 2+4/3+4, 8+7/10+5, 5+3/6+6, and 6+7/7+5 programs of GCC/LLVM, respectively. This observation confirms that CompTuner outperforms the baselines in selecting a flag combination with good performance in most new cases.

Table 19. Statistics of Programs

ID	Program	Lines	ID	Program	Lines
P11	gemm	221	P21	mvt	210
P12	gemver	249	P22	durbin	195
P13	gesummv	210	P23	gramschmidt	220
P14	seidel-2d	179	P24	ludcmp	247
P15	syr2k	214	P25	trisolv	183
P16	syrk	199	P26	deriche	265
P17	trmm	199	P27	floyd-warshall	176
P18	atax	198	P28	adi	237
P19	bicg	214	P29	fdtd-2d	245
P20	doitgen	203	P30	jacobi-1d	186
C11	bzip2d	7,200	C20	security_rijndael_d	1,799
C12	consumer_jpeg_d	26,183	C21	security_rijndael_e	1,799
C13	consumer_lame	21,824	C22	office_stringsearch1	508
C14	consumer_mad	25,863	C23	security_blowfish_d	1,524
C15	consumer_tiff2bw	22,271	C24	security_blowfish_e	1,524
C16	consumer_tiffdither	22,184	C25	telecom_adpcm_d	391
C17	consumer_tiffmedian	22,756	C26	telecom_CRC32	307
C18	network_dijkstra	199	C27	telecom_gsm	6,049
C19	network_patricia	634	C28	automotive_qsort1	227

Table 20. Performance Improvement for Programs of PolyBench over -O3 on GCC

Technique	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup
CompTuner		1.051(3,834)		1.184(3,501)		1.128(4,496)		1.073(5,058)		1.092(2,653)
RIO		✗		✗		✗		1.084(5,573)		✗
GA		1.074(2,712)		✗		✗		✗		✗
TPE	P11	✗	P12	1.188(5,649)	P13	1.149(2,577)	P14	✗	P15	✗
BOCA		1.063(3,139)		1.129(4,719)		✗		1.088(3,438)		✗
OpenTuner		1.077(4,438)		1.188(3,999)		✗		1.079(4,381)		✗
COBYAN		✗		✗		1.136(5,157)		✗		1.103(4,897)
CompTuner		1.093(4,598)		1.138(3,650)		1.058(2,932)		1.205(4,934)		1.066(3,471)
RIO		1.099(5,847)		✗		✗		1.218(2,929)		✗
GA		✗		1.147(4,559)		✗		✗		1.078(3,997)
TPE	P16	✗	P17	1.142(4,826)	P18	✗	P19	1.207(3,762)	P20	1.082(4,565)
BOCA		1.143(3,651)		✗		1.042(3,664)		1.212(4,138)		1.105(2,916)
OpenTuner		✗		1.141(3,947)		✗		✗		1.068(4,013)
COBYAN		1.105(3,792)		1.152(3,595)		✗		✗		✗
CompTuner		1.111(3,953)		1.058(2,659)		1.078(4,452)		1.029(3,549)		1.105(3,674)
RIO		✗		1.062(3,575)		✗		1.041(3,836)		✗
GA		1.124(4,743)		✗		1.083(4,930)		✗		✗
TPE	P21	✗	P22	✗	P23	1.093(2,989)	P24	✗	P25	1.113(4,487)
BOCA		1.119(4,502)		1.084(2,273)		✗		1.075(3,108)		1.109(3,782)
OpenTuner		1.127(3,205)		✗		1.079(2,680)		1.037(4,471)		✗
COBYAN		✗		1.072(2,909)		✗		1.091(3,170)		1.117(3,103)
CompTuner		1.101(3,832)		1.795(5,118)		1.038(4,327)		1.057(3,731)		1.104(2,132)
RIO		✗		✗		1.045(4,959)		✗		✗
GA		✗		1.845(3,902)		✗		1.062(3,237)		1.123(2,925)
TPE	P26	1.115(4,875)	P27	✗	P28	✗	P29	1.059(4,851)	P30	✗
BOCA		1.128(5,157)		1.799(4,135)		✗		✗		1.115(3,989)
OpenTuner		1.109(3,135)		1.872(4,739)		1.062(5,082)		1.072(4,136)		✗
COBYAN		1.117(3,358)		✗		1.049(4,058)		1.081(5,726)		1.129(3,846)

To sum up, the flag combination selected by our proposed technique CompTuner outperforms the default -O3 optimization of GCC and LLVM. Moreover, in most cases, CompTuner outperforms the compared compiler autotuning techniques (including the state-of-the-art BOCA), demonstrating that CompTuner is promising.

Table 21. Performance Improvement for Programs of PolyBench over -O3 on LLVM

Technique	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup
CompTuner		1.088(3,862)		1.119(3,015)		1.073(4,480)		1.072(4,127)		1.085(3,152)
RIO		✗		1.154(3,897)		✗		✗		1.087(4,463)
GA		1.093(4753)		✗		1.079(2,937)		✗		✗
TPE	P11	1.088(3,104)	P12	✗	P13	1.092(3,471)	P14	1.077(4,992)	P15	✗
BOCA		1.097(3,573)		✗		✗		1.091(3,382)		1.105(3,872)
OpenTuner		✗		1.126(4,717)		✗		✗		1.092(3,378)
COBYAN		✗		1.121(3,937)		1.077(4,106)		1.103(4,792)		✗
CompTuner		1.063(4,718)		1.030(5,536)		1.068(4,901)		1.035(3,834)		1.149(4,091)
RIO		1.068(5,937)		✗		1.079(4,105)		✗		✗
GA		✗		1.032(3,832)		1.090(4,569)		✗		✗
TPE	P16	✗	P17	1.039(3,971)	P18	✗	P19	✗	P20	1.167(3,058)
BOCA		1.084(5,106)		1.049(3,137)		1.072(4,275)		1.051(2,968)		✗
OpenTuner		✗		1.064(3,218)		✗		1.039(4,252)		✗
COBYAN		✗		✗		1.069(3,761)		1.044(4,471)		1.173(2,968)
CompTuner		1.211(3,382)		1.186(4,571)		1.067(3,152)		1.072(3,974)		1.232(3,162)
RIO		✗		✗		1.074(4,836)		✗		✗
GA		1.149(3,121)		✗		✗		1.069(4,376)		✗
TPE	P21	✗	P22	✗	P23	1.072(3,062)	P24	✗	P25	✗
BOCA		✗		1.235(3,638)		1.085(4,579)		✗		✗
OpenTuner		✗		✗		✗		1.087(3,068)		1.265(2,950)
COBYAN		✗		1.192(4,959)		1.081(3,768)		1.083(3,520)		✗
CompTuner		1.059(3,727)		1.028(4,093)		1.072(4,485)		1.036(4,132)		1.193(3,568)
RIO		✗		✗		✗		1.058(5,712)		✗
GA		1.061(4,593)		✗		✗		✗		1.196(6,518)
TPE	P26	✗	P27	1.034(3,902)	P28	1.082(4,139)	P29	✗	P30	✗
BOCA		✗		✗		1.109(4,885)		1.067(3,461)		1.204(4,957)
OpenTuner		✗		1.059(3,268)		1.077(5,386)		✗		✗
COBYAN		1.073(3,219)		1.030(3,364)		1.096(3,902)		✗		1.223(4,152)

Table 22. Performance Improvement for Programs of cBench over -O3 on GCC

Technique	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup
CompTuner		1.084(4,172)		1.047(2,439)		1.056(5,102)		1.077(3,562)		1.088(4,574)
RIO		✗		✗		1.072(2,271)		✗		✗
GA		✗		✗		✗		✗		✗
TPE	C11	✗	C12	✗	C13	✗	C14	✗	C15	✗
BOCA		1.093(3,351)		1.055(4,843)		1.063(5,594)		✗		✗
OpenTuner		✗		✗		✗		1.079(4,471)		✗
COBYAN		1.089(5,059)		✗		1.072(4,165)		✗		1.077(5,641)
CompTuner		1.122(5,032)		1.093(2,627)		1.071(3,368)		1.102(4,718)		1.065(4,179)
RIO		✗		✗		1.099(4,102)		✗		1.082(4,432)
GA		✗		✗		✗		✗		1.069(3,832)
TPE	C16	✗	C17	1.094(3,912)	C18	✗	C19	✗	C20	✗
BOCA		✗		✗		✗		1.133(3,672)		1.077(4,780)
OpenTuner		✗		1.102(4,563)		✗		✗		1.112(2,919)
COBYAN		1.159(3,176)		✗		1.078(2,729)		1.105(5,102)		1.094(3,587)
CompTuner		1.083(3,602)		1.137(5,118)		1.255(4,565)		1.116(3,117)		1.084(3,058)
RIO		✗		✗		✗		✗		1.088(2,271)
GA		✗		✗		✗		✗		1.093(4,603)
TPE	C21	1.089(2,844)	C22	✗	C23	✗	C24	✗	C25	✗
BOCA		✗		1.118(4,659)		1.293(3,364)		1.127(3,954)		✗
OpenTuner		✗		✗		1.362(4,982)		1.133(2,732)		✗
COBYAN		✗		✗		✗		✗		1.102(4,120)
CompTuner		1.097(2,281)		1.121(4,774)		1.076(3,953)				
RIO		✗		✗		✗				
GA		✗		✗		✗				
TPE	C26	1.099(4,574)	C27	✗	C28	1.089(4,421)				
BOCA		1.113(3,029)		1.133(5,549)		1.115(2,984)				
OpenTuner		✗		✗		✗				
COBYAN		✗		1.125(3,328)		1.103(4,015)				

Table 23. Performance Improvement for Programs of cBench over -O3 on LLVM

Technique	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup	ID	Speedup
CompTuner		1.154(3,458)		1.071(4,972)		1.063(3,079)		1.057(4,551)		1.137(4,018)
RIO		1.162(4,619)		✗		✗		✗		✗
GA		✗		1.079(2,271)		1.082(4,637)		✗		✗
TPE	C11	✗	C12	✗	C13	✗	C14	1.077(3,268)	C15	✗
BOCA		1.189(2,754)		1.075(3,884)		1.066(3,159)		✗		1.158(3,792)
OpenTuner		✗		1.103(3,479)		✗		1.059(3,821)		✗
COBYAN		1.171(3,912)		✗		✗		✗		1.139(5,631)
CompTuner		1.108(3,419)		1.162(4,728)		1.086(5,593)		1.038(3,602)		1.059(4,896)
RIO		✗		✗		1.092(3,874)		1.039(3,913)		✗
GA		1.114(4,264)		✗		✗		✗		1.074(4,152)
TPE	C16	✗	C17	1.173(5,816)	C18	✗	C19	✗	C20	1.096(4,428)
BOCA		1.142(3,927)		✗		1.114(3,167)		✗		✗
OpenTuner		1.108(3,476)		1.187(4,105)		✗		1.045(2,932)		✗
COBYAN		✗		1.179(5,182)		✗		1.088(3,990)		1.072(2,864)
CompTuner		1.223(3,652)		1.091(2,958)		1.116(3,971)		1.108(3,259)		1.058(3,168)
RIO		✗		✗		✗		✗		1.071(3,964)
GA		1.245(4,576)		1.134(4,048)		✗		✗		✗
TPE	C21	✗	C22	1.097(3,682)	C23	✗	C24	✗	C25	✗
BOCA		1.264(3,057)		1.115(3,364)		✗		✗		1.073(3,019)
OpenTuner		✗		✗		1.155(3,168)		1.085(3,554)		1.092(4,083)
COBYAN		✗		1.152(3,794)		1.126(3,091)		✗		1.059(2,845)
CompTuner		1.046(3,782)		1.179(4,861)		1.097(3,752)				
RIO		✗		✗		✗				
GA		✗		1.185(4,013)		✗				
TPE	C26	✗	C27	1.180(3,894)	C28	✗				
BOCA		✗		1.194(3,265)		1.112(4,183)				
OpenTuner		✗		✗		1.135(4,416)				
COBYAN		1.046(3,952)		✗		1.158(3,583)				

Table 24. Number of Top-2 Performance Results

Technique	GCC	LLVM	All
CompTuner	11/15	10/13	21/28
RIO	3/1	0/3	3/4
GA	3/1	2/2	5/3
TPE	2/4	3/4	5/8
BOCA	8/7	10/5	18/12
OpenTuner	5/3	6/6	11/9
COBYAN	6/7	7/5	13/12

REFERENCES

- [1] Felix V. Agakov, Edwin V. Bonilla, John Cavazos, Björn Franke, Grigori Fursin, Michael F. P. O'Boyle, John Thomson, Marc Toussaint, and Christopher K. I. Williams. 2006. Using machine learning to focus iterative optimization. In *4th IEEE/ACM International Symposium on Code Generation and Optimization (CGO'06)*. IEEE Computer Society, 295–305.
- [2] Lelac Almador, Keith D. Cooper, Alexander Grosul, Timothy J. Harvey, Steven W. Reeves, Devika Subramanian, Linda Torczon, and Todd Waterman. 2004. Finding effective compilation sequences. *ACM SIGPLAN Notices* 39, 7 (2004), 231–239.
- [3] Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Una-May O'Reilly, and Saman P. Amarasinghe. 2014. OpenTuner: An extensible framework for program autotuning. In *International Conference on Parallel Architectures and Compilation (PACT'14)*, José Nelson Amaral and Josep Torrellas (Eds.). ACM, 303–316.
- [4] Amir Hossein Ashouri. 2016. *Compiler Autotuning Using Machine Learning Techniques*. Ph.D. thesis, 3–4.
- [5] Amir H. Ashouri, Andrea Bignoli, Gianluca Palermo, Cristina Silvano, Sameer Kulkarni, and John Cavazos. 2017. Micomp: Mitigating the compiler phase-ordering problem using optimization sub-sequences and machine learning. *ACM Transactions on Architecture and Code Optimization (TACO)* 14, 3 (2017), 1–28.
- [6] Amir H. Ashouri, William Killian, John Cavazos, Gianluca Palermo, and Cristina Silvano. 2018. A survey on compiler autotuning using machine learning. *ACM Computing Surveys (CSUR)* 51, 5 (2018), 1–42.
- [7] Amir Hossein Ashouri, Giovanni Mariani, Gianluca Palermo, Eunjung Park, John Cavazos, and Cristina Silvano. 2016. COBAYN: Compiler autotuning framework using bayesian networks. *ACM Transactions on Architecture and Code Optimization* 13, 2, Article 21 (June 2016), 25 pages.
- [8] Amir H. Ashouri, Gianluca Palermo, John Cavazos, and Cristina Silvano. 2018. *Automatic Tuning of Compilers Using Machine Learning*. Springer.
- [9] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. 2011. Algorithms for hyper-parameter optimization. *Advances in Neural Information Processing Systems* 24 (2011), 2546–2554.
- [10] John Cavazos, Grigori Fursin, Felix Agakov, Edwin Bonilla, Michael F. P. O'Boyle, and Olivier Temam. 2007. Rapidly selecting good compiler optimizations using performance counters. In *International Symposium on Code Generation and Optimization (CGO'07)*. IEEE, 185–197.
- [11] cBench. 2022. <https://ctuning.org/wiki/index.php/CTools:CBench>
- [12] Junjie Chen, Ningxin Xu, Peiqi Chen, and Hongyu Zhang. 2021. Efficient compiler autotuning via Bayesian optimization. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE'21)*. IEEE, 1198–1209.
- [13] Yang Chen, Shuangde Fang, Yuanjie Huang, Lieven Eeckhout, Grigori Fursin, Olivier Temam, and Chengyong Wu. 2012. Deconstructing iterative optimization. *ACM Transactions on Architecture and Code Optimization (TACO)* 9, 3 (2012), 1–30.
- [14] Chris Cummins, Pavlos Petoumenos, Zheng Wang, and Hugh Leather. 2017. End-to-end deep learning of optimization heuristics. In *26th International Conference on Parallel Architectures and Compilation Techniques (PACT'17)*. IEEE Computer Society, 219–232.
- [15] Oguzhan Dogru, Kirubakaran Velswamy, Fadi Ibrahim, Yuqi Wu, Arun Senthil Sundaramoorthy, Biao Huang, Shu Xu, Mark Nixon, and Noel Bell. 2022. Reinforcement learning approach to autonomous PID tuning. *Computers & Chemical Engineering* 161 (2022), 107760.
- [16] Qi Du and Hui Huang. 2020. Research on the realization and optimization of FFTs in ARMv8 platform. In *IOP Conference Series: Materials Science and Engineering*, Vol. 768. IOP Publishing, 072114.
- [17] Qi Du, Hui Huang, and Chun Huang. 2020. Roofline model-guided compilation optimization parameter selection method. In *Proceedings of Data Science: 6th International Conference of Pioneering Computer Scientists, Engineers and Educators (ICPCSEE'20), Part I* 6. Springer, 268–282.
- [18] Alexis Engelke and Martin Schulz. 2020. Robust practical binary optimization at run-time using LLVM. In *2020 IEEE/ACM 6th Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC'20) and Workshop on Hierarchical Parallelism for Exascale Computing (HiPar'20)*. IEEE, 56–64.
- [19] Matteo Fischetti and Matteo Stringher. 2019. Embedded hyper-parameter tuning by Simulated Annealing. *CoRR* abs/1906.01504. arXiv:1906.01504 <http://arxiv.org/abs/1906.01504>
- [20] Peter I. Frazier. 2018. A tutorial on Bayesian optimization. *arXiv preprint arXiv:1807.02811*.
- [21] Unai Garciaarena and Roberto Santana. 2016. Evolutionary optimization of compiler flag selection by learning and exploiting flags interactions. In *Proceedings of the 2016 on Genetic and Evolutionary Computation Conference Companion*. 1159–1166.
- [22] GCC. 2022. <https://gcc.gnu.org>
- [23] Ameer Haj-Ali, Nesreen K. Ahmed, Theodore L. Willke, Yakun Sophia Shao, Krste Asanovic, and Ion Stoica. 2020. NeuroVectorizer: End-to-end vectorization with deep reinforcement learning. In *18th ACM/IEEE International Symposium on Code Generation and Optimization (CGO'20)*. ACM, 242–255.

- [24] Haochen He, Zhouyang Jia, Shanshan Li, Yue Yu, Chenglong Zhou, Qing Liao, Ji Wang, and Xiangke Liao. 2022. Multi-intention-aware configuration selection for performance tuning. In *44th IEEE/ACM 44th International Conference on Software Engineering (ICSE'22)*. ACM, 1431–1442.
- [25] Christopher Henard, Mike Papadakis, Mark Harman, and Yves Le Traon. 2015. Combining multi-objective search and constraint solving for configuring large software product lines. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. 517–528.
- [26] Kenneth Hoste and Lieven Eeckhout. 2008. Cole: Compiler optimization level exploration. In *Proceedings of the 6th Annual IEEE/ACM International Symposium on Code Generation and Optimization*. 165–174.
- [27] Qijing Huang, Ameer Haj-Ali, William Moses, John Xiang, Ion Stoica, Krste Asanovic, and John Wawrzynek. 2019. Autophase: Compiler phase-ordering for hls with deep reinforcement learning. In *2019 IEEE 27th Annual International Symposium on Field-programmable Custom Computing Machines (FCCM'19)*. IEEE, 308–308.
- [28] Michael R. Jantz and Prasad A. Kulkarni. 2013. Performance potential of optimization phase selection during dynamic JIT compilation. In *ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (co-located with ASPLOS'13) (VEE'13)*, Steve Muir, Gernot Heiser, and Stephen M. Blackburn (Eds.). ACM, 131–142.
- [29] James Kennedy and Russell Eberhart. 1995. Particle swarm optimization. In *Proceedings of International Conference on Neural Networks (ICNN'95)*, Vol. 4. IEEE, 1942–1948.
- [30] Adam Lipowski and Dorota Lipowska. 2012. Roulette-wheel selection via stochastic acceptance. *Physica A: Statistical Mechanics and Its Applications* 391, 6 (2012), 2193–2196.
- [31] LLVM. 2022. <https://llvm.org>
- [32] Antoine Monsifrot, François Bodin, and René Quiniou. 2002. A machine learning approach to automatic production of compiler heuristics. In *International Conference on Artificial Intelligence: Methodology, Systems, and Applications*. Springer, 41–50.
- [33] Vivek Nair, Zhe Yu, Tim Menzies, Norbert Siegmund, and Sven Apel. 2020. Finding faster configurations using FLASH. *IEEE Transactions on Software Engineering* 46, 7 (2020), 794–811.
- [34] Sajjad Nematzadeh, Farzad Kiani, Mahsa Torkamanian-Afshar, and Nizamettin Aydin. 2022. Tuning hyperparameters of machine learning algorithms and deep neural networks using metaheuristics: A bioinformatics study on biomedical and biological cases. *Computational Biology and Chemistry* 97 (2022), 107619.
- [35] Youcong Ni, Rui Wu, Xin Du, Peng Ye, Wangbiao Li, and Ruliang Xiao. 2019. Evolutionary algorithm for optimization of energy consumption at GCC compile time based on frequent pattern mining. *Journal of Software* 30, 5 (2019), 1269–1287.
- [36] Numpy. 2022. <https://numpy.org>
- [37] David A. Padua and Michael J. Wolfe. 1986. Advanced compiler optimizations for supercomputers. *Communications of the ACM* 29, 12 (1986), 1184–1201.
- [38] Juliana Alves Pereira, Mathieu Acher, Hugo Martin, and Jean-Marc Jézéquel. 2020. Sampling effect on performance prediction of configurable systems: A case study. In *11th ACM/SPEC International Conference on Performance Engineering (ICPE'20)*. ACM, 1–13.
- [39] Leslie Pérez Cáceres, Federico Pagnozzi, Alberto Franzin, and Thomas Stützle. 2017. Automatic configuration of GCC using Irace. In *International Conference on Artificial Evolution (Evolution Artificielle)*. Springer, 202–216.
- [40] Riccardo Poli, James Kennedy, and Tim Blackwell. 2007. Particle swarm optimization. *Swarm Intelligence* 1, 1 (2007), 33–57.
- [41] PolyBench. 2022. <https://web.cse.ohio-state.edu/~pouchet.2/software/polybench/>
- [42] Suresh Purini and Lakshya Jain. 2013. Finding good optimization sequences covering program space. *ACM Transactions on Architecture and Code Optimization (TACO)* 9, 4 (2013), 1–23.
- [43] Thayalan Sandran, Nordin Zakaria, and Anindya Jyoti Pal. 2012. An optimized tuning of genetic algorithm parameters in compiler flag selection based on compilation and execution duration. In *Proceedings of the International Conference on Soft Computing for Problem Solving (SocProS'11)*. Springer, 599–610.
- [44] Chompunuch Sarasaen, Soumick Chatterjee, Mario Breitkopf, Georg Rose, Andreas Nürnberger, and Oliver Speck. 2021. Fine-tuning deep learning model parameters for improved super-resolution of dynamic MRI with prior-knowledge. *CoRR* abs/2102.02711. arXiv:2102.02711 <https://arxiv.org/abs/2102.02711>
- [45] Paul B. Schneek. 1973. A survey of compiler optimization techniques. In *Proceedings of the ACM Annual Conference*. 106–113.
- [46] scikit learn. 2022. <https://scikit-learn.org/stable/>
- [47] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando De Freitas. 2015. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE* 104, 1 (2015), 148–175.
- [48] Yuhui Shi and Russell C. Eberhart. 1999. Empirical study of particle swarm optimization. In *Proceedings of the 1999 Congress on Evolutionary Computation (CEC'99) (Cat. No. 99TH8406)*, Vol. 3. IEEE, 1945–1950.

- [49] Norbert Siegmund, Alexander Grebhahn, Sven Apel, and Christian Kästner. 2015. Performance-influence models for highly configurable systems. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 284–294.
- [50] Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. 2012. Practical Bayesian optimization of machine learning algorithms. *Advances in Neural Information Processing Systems* 25 (2012), 2951–2959.
- [51] Ananta Tiwari, Chun Chen, Jacqueline Chame, Mary Hall, and Jeffrey K. Hollingsworth. 2009. A scalable auto-tuning framework for compiler optimization. In *2009 IEEE International Symposium on Parallel & Distributed Processing*. IEEE, 1–12.
- [52] Ruben Vazquez, Ann Gordon-Ross, and Greg Stitt. 2019. Energy prediction for cache tuning in embedded systems. In *37th IEEE International Conference on Computer Design (ICCD'19)*. IEEE, 630–637.
- [53] Helen Victoria and Ganesh Maragatham. 2021. Automatic tuning of hyperparameters using Bayesian optimization. *Evolving Systems* 12 (2021), 12–30.
- [54] Yu Wang, Shu Jiang, Weiman Lin, Yu Cao, Longtao Lin, Jiangtao Hu, Jinghao Miao, and Qi Luo. 2021. A learning-based automatic parameters tuning framework for autonomous vehicle control in large scale system deployment. In *2021 American Control Conference (ACC'21)*. IEEE, 2919–2926.
- [55] Jun Ye. 2011. Cosine similarity measures for intuitionistic fuzzy sets and their applications. *Mathematical and Computer Modelling* 53, 1–2 (2011), 91–97.

Received 28 October 2022; revised 18 December 2023; accepted 27 December 2023